

第3章 布尔代数基础

布尔代数得名于英国数学家乔治·布尔(George Boole, 1815—1864)。为了研究人的逻辑思维规律,他在1847年发表的《逻辑的数学分析》和1854年发表的《思维规律的研究》两部著作中,首先提出了这种代数的基本概念和性质。此后,大约经过近100年之久,于1938年才由克劳德·香农(Claude E. Shannon)将布尔代数应用于电话继电器的开关电路中。至今,布尔代数已成为分析和设计开关电路的重要数学工具。

本章不是从数学的角度去研究布尔代数,而是从应用的角度介绍布尔代数的一些基本概念、基本定理、布尔函数的基本形式以及布尔函数的化简方法,以使读者掌握分析和设计数字逻辑网络所需的数学工具。

3.1 布尔代数的基本概念

计算机或其他数字系统无论多么复杂,它们都是由若干种最简单的、最基本的电路(如门电路、触发器等)所组成的。这些电路的工作具有下列基本特点:从电路内部看,或是管子导通,或是管子截止;从电路的输入输出看,或是电平的高低,或是脉冲的有无。由于这种电路工作在开关状态,故称为开关电路。开关电路的工作状态可以用二元布尔代数来描述,故二元布尔代数通常称为开关代数,或称为逻辑代数。因此,逻辑代数只是布尔代数的一种特例。在本书中,如无特别说明,布尔代数均指逻辑代数。

3.1.1 布尔变量及其基本运算

布尔代数和普通代数一样,用字母代表变量,布尔代数的变量称为布尔变量。和普通代数不同的是,布尔变量只有两种取值,即0或1。并且,常量0和1没有普通代数中的0和1的意义,它只表示两种可能,即命题的“假”和“真”,信号的“无”和“有”等。

布尔代数中的变量运算只有“或”、“与”、“非”3种基本运算,任何复杂的逻辑运算都可以通过这3种基本运算来实现。

1. “或”运算

“或”运算又称为逻辑加。两个变量“或”运算的逻辑关系可表示为

$$F = A + B$$

式中,“+”号是“或”运算符。上式读作“F等于A或B”,或者“F等于A加B”,其意思是变量A和B中只要有一者取值为1,则F就为1;若A和B全为0,则F为0。其逻辑关系可以用真值表来描述,如表3.1所示。

2. “与”运算

“与”运算又称为逻辑乘。两个变量的“与”运算的逻辑关系可表示为

$$F = A \cdot B$$

式中“ $\cdot$ ”号表示“与”运算符。通常，“与”运算符可以省略。上式读作“ F 等于 A 与 B ”，或者“ F 等于 A 乘 B ”。其含义是只有当变量 A 与 B 都为 1 时； F 才为 1；否则， F 就为 0。其逻辑关系可以用真值表来描述，如表 3.2 所示。

表 3.1 “或”运算

A	B	F
0	0	0
0	1	1
1	0	1
1	1	1

表 3.2 “与”运算

A	B	F
0	0	0
0	1	0
1	0	0
1	1	1

3. “非”运算

“非”运算又称为逻辑取反。对一个变量的“非”运算的逻辑关系可表示为：

$$F = \bar{A}$$

式中“ $\bar{}$ ”号表示“非”运算符。上式读作“ F 等于 A 的非”，其意思是若 A 为 1，则 F 为 0；反之，若 A 为 0，则 F 为 1。“非”运算的逻辑关系可以用表 3.3 所示的真值表来描述。

表 3.3 “非”运算

A	F
0	1
1	0

综合上述对布尔变量及其 3 个基本运算的定义，我们可以对布尔代数下个定义：

布尔代数是一个由布尔变量集 K ，常量 0、1 以及“或”、“与”、“非”3 种运算符所构成的代数系统，记为

$$B = (K, +, \cdot, -, 0, 1)$$

其中，布尔变量集 K 是指布尔代数中的所有可能变量的集合，它可用任何字母表示，但每一个变量的取值只可能为常量 0 或 1，而且布尔代数中的变量只有“或”、“与”、“非”3 种运算。

3.1.2 布尔函数及其表示方法

布尔代数中的函数定义与普通代数中函数定义十分相似，可以叙述如下。

设 $(x_1, x_2, \dots, x_n)$ 为布尔代数的一组布尔变量，其中每个变量取值为 0 或 1，则当把 n 序列 $(x_1, x_2, \dots, x_n)$ 映射到 $B = \{0, 1\}$ 时，这个映射就是一个布尔函数。

从另一个角度，把布尔函数与逻辑网络联系起来，布尔函数可以这样叙述：

设某一逻辑网络的输入变量为 $x_1, x_2, \dots, x_n$ ，输出变量为 F ，如图 3.1 所示。对应于变量 $x_1, x_2, \dots, x_n$ 的每一组确定值， F 就有唯一确定的值，则称 F 是变量 $x_1, x_2, \dots, x_n$ 的布尔函数。记为



图 3.1 布尔函数

$$F = f(x_1, x_2, \dots, x_n)$$

注意,布尔代数中函数的取值也只可能是 0 或 1,这与普通代数是不同的。

布尔函数的表示方法有 3 种形式:布尔表达式、真值表和卡诺图。这与普通代数中用公式、表格和图解这 3 种方法来表示函数十分类似。

1. 布尔表达式

布尔表达式是由布尔变量和“或”、“与”、“非”3种运算符所构成的式子,这是一种用公式表示布尔函数的方法。例如,要表示这样一个函数关系:当两个变量 A 和 B 取值相同时,函数取值为 0;否则,函数取值为 1。此函数称为异或函数,可以用下列布尔表达式来表示:

$$F = f(A, B) = \bar{A}B + A\bar{B}$$

显然,只要将 A 和 B 的 4 种可能取值代入这表达式,验证是正确的。

与异或函数相反,当两个变量 A 和 B 取值相同时,函数取值为 1;否则,函数取值为 0。此函数称为同或函数。通常,异或运算用符号 $\oplus$ 表示;同或运算用 $\odot$ 表示。因此,异或函数、同或函数可分别表示成:

$$F = \bar{A}B + A\bar{B} = A \oplus B; F = \bar{A}\bar{B} + AB = A \odot B$$

2. 真值表

真值表是由输入变量的所有可能取值组合及其对应的输出函数值所构成的表格,这是一种用表格表示布尔函数的方法。例如,对于前面的异或函数,可以用表 3.4 所示的真值表来表示。

真值表中的变量为两个,共有 2^2 种取值组合,所以该表由 4 行组成。当变量为 n 个时,真值表就由 2^n 行组成。显然,随着变量数目的增加,真值表的行数将急剧增加。因此,一般当变量数目不超过 4 个时,用真值表表示函数比较方便。

表 3.4 异或函数的真值表

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

3. 卡诺图

卡诺图是由表示逻辑变量的所有可能取值组合的小方格所构成的图形,如图 3.2 所示。图中分别表示了二变量及三变量的卡诺图。

利用卡诺图,可以很方便地表示一个函数。只要在这些使函数值为 1 的变量取值组合

所对应的小方格上标记 1, 便得该函数的卡诺图。例如, 对于异或函数, 可以用图 3.3 所示的卡诺图来表示。

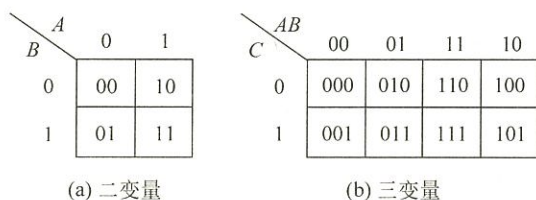


图 3.2 卡诺图

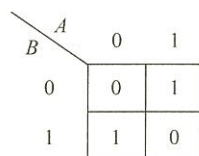


图 3.3 异或函数的卡诺图

卡诺图可以看成是真值表的重新排列, 真值表的每一行用一个小方格来表示。当变量为两个时, 真值表有 4 行, 相应的卡诺图有 4 个方格; 当变量为 n 个时, 卡诺图有 2^n 个方格。卡诺图的这种方格排列方式比真值表更紧凑, 而且便于进行函数的简化。

3.1.3 布尔函数的“相等”概念

布尔函数和普通代数一样, 也有函数相等的问题。两个函数相等的定义如下。

设有两个布尔函数

$$F = f(x_1, x_2, \dots, x_n)$$

$$G = g(x_1, x_2, \dots, x_n)$$

其变量都为 $x_1, x_2, \dots, x_n$ 。如果对应于变量 $x_1, x_2, \dots, x_n$ 的任何一组变量取值, F 和 G 的值都相同, 则称 F 和 G 是相等的, 记为 $F=G$ 。

显然, 若两个布尔函数相等, 则它们的真值表一定相同; 反之, 若两个布尔函数的真值表完全相同, 则此两个函数相等。因此, 要证明两个布尔函数是否相等, 只要分别列出它们的真值表, 看其是否相同。

例如, 已知下列两个函数

$$F = \overline{xy} \quad G = \bar{x} + \bar{y}$$

列出 F 和 G 的真值表, 如表 3.5 所示。由表可知, 它们的真值表完全相同, 故 F 和 G 是相等的, 即有

$$\overline{xy} = \bar{x} + \bar{y}$$

表 3.5 $F=\overline{xy}$ 和 $G=\bar{x}+\bar{y}$ 的真值表

x	y	xy	$F=\overline{xy}$	$\bar{x}$	$\bar{y}$	$G=\bar{x}+\bar{y}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

$$A \cdot 1 = A$$

$$: \text{左} = A \cdot 1 + A \cdot B = A \cdot (1 + B) = A \cdot 1 = A = \text{右}$$

3.2 布尔代数的公式、定理和规则

3.2.1 布尔代数的基本公式

根据布尔变量的取值只有0和1,以及布尔变量仅有的3种运算的定义,不难推出下列基本公式。

(1) 交换律

$$A + B = B + A$$

$$A \cdot B = B \cdot A$$

(2) 结合律

$$(A + B) + C = A + (B + C)$$

$$(A \cdot B) \cdot C = A \cdot (B \cdot C)$$

(3) 分配律

$$A \cdot (B + C) = A \cdot B + A \cdot C$$

$$A + B \cdot C = (A + B)(A + C)$$

(4) 0-1 律

$$\begin{cases} A + 0 = A \\ A + 1 = 1 \\ A \cdot 0 = 0 \\ A \cdot 1 = A \end{cases}$$

(5) 互补律

$$A + \bar{A} = 1$$

$$A \cdot \bar{A} = 0$$

(6) 等幂律

$$A + A = A$$

$$A \cdot A = A$$

(7) 吸收律

$$\begin{cases} A + AB = A \\ A + \bar{A}B = A + B \\ A(A + B) = A \\ A(\bar{A} + B) = AB \end{cases}$$

(8) 对合律(双重否定律)

$$\bar{\bar{A}} = A$$

以上是布尔代数的基本公式。其中交换律、结合律、分配律、0-1律、互补律和对合律可以作为布尔代数的公理。公理是代数系统的基本出发点,是客观存在的抽象,它无须证明,

但它可以用客观存在来验证。以此为基础,可以推得布尔代数的等幂律与吸收律。例如,等幂律的证明如下:

$$\begin{aligned}
 A + A &= (A + A) \cdot 1 && (0-1 \text{ 律}) \\
 &= (A + A)(A + \bar{A}) && (\text{互补律}) \\
 &= A + A \cdot \bar{A} && (\text{分配律}) \\
 &= A + 0 && (\text{互补律}) \\
 &= A && (0-1 \text{ 律})
 \end{aligned}$$

又如,吸收律的证明如下:

$$\begin{aligned}
 A + \bar{A}B &= (A + \bar{A})(A + B) && (\text{分配律}) \\
 &= 1 \cdot (A + B) && (\text{互补律}) \\
 &= A + B && (0-1 \text{ 律})
 \end{aligned}$$

必须指出,上述基本公式中,有些公式与普通代数中的相同,如交换律、结合律,但有些公式却是布尔代数中所特有的,如分配律。

3.2.2 布尔代数的主要定理

定理 1 德·摩根(De Morgan)定理。

$$\begin{cases}
 (1) \overline{x_1 + x_2 + \cdots + x_n} = \bar{x}_1 \cdot \bar{x}_2 \cdot \cdots \cdot \bar{x}_n \\
 (2) \overline{x_1 \cdot x_2 \cdot \cdots \cdot x_n} = \bar{x}_1 + \bar{x}_2 + \cdots + \bar{x}_n
 \end{cases}$$

这就是说, n 个变量的“或”的“非”等于各变量的“非”的“与”, n 个变量的“与”的“非”等于各变量的“非”的“或”。

当变量数目较少时,该定理可很容易用真值表证明。当变量为 n 个时,则可以用数学归纳法证明。

德·摩根定理是布尔代数中一个很重要、且经常使用的定理,它提供了一种变换布尔表达式的简便方法。由于它具有反演特性,即把变量的与运算改成或运算,或运算改成与运算,所以又称为反演律。

定理 2 香农(Shannon)定理。

$$\overline{f(x_1, x_2, \cdots, x_n, 0, 1, +, \cdot)} = f(\bar{x}_1, \bar{x}_2, \cdots, \bar{x}_n, 1, 0, \cdot, +)$$

这就是说,任何函数的反函数(或称补函数),可以通过对该函数的所有变量取反,并将常量1换为0,0换为1,运算符“+”换为“ $\cdot$ ”,“ $\cdot$ ”换为“+”而得到。

证明 根据德·摩根定理,任何函数的反函数可写成

$$\begin{aligned}
 &\overline{f(x_1, x_2, \cdots, x_n, 0, 1, +, \cdot)} \\
 &= \overline{f_1(x_1, x_2, \cdots, x_n, 0, 1, +, \cdot) + f_2(x_1, x_2, \cdots, x_n, 0, 1, +, \cdot)} \\
 &= \overline{f_1(x_1, x_2, \cdots, x_n, 0, 1, +, \cdot)} \cdot \overline{f_2(x_1, x_2, \cdots, x_n, 0, 1, +, \cdot)}
 \end{aligned}$$

或写成:

$$\begin{aligned}
 &\overline{f(x_1, x_2, \cdots, x_n, 0, 1, +, \cdot)} \\
 &= \overline{f_1(x_1, x_2, \cdots, x_n, 0, 1, +, \cdot)} \cdot \overline{f_2(x_1, x_2, \cdots, x_n, 0, 1, +, \cdot)} \\
 &= \overline{f_1(x_1, x_2, \cdots, x_n, 0, 1, +, \cdot)} + \overline{f_2(x_1, x_2, \cdots, x_n, 0, 1, +, \cdot)}
 \end{aligned}$$

其中 f_1 和 f_2 是 f 的两个部分函数。对 f_1 和 f_2 重复上述过程,直到使 f 中的每个变量都用德·摩根定理。由于每对 f (或 f 的部分函数)应用一次德·摩根定理,就将部分函数(或子部分函数)取反,并将“与”、“或”运算变换一次,以求得函数 f (或部分函数)的反函数 $\bar{f}$,因此,当对 f 的每个变量进行德·摩根变换后,其结果必然是 $f(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n, 1, 0, \dots, +)$, 证毕。

香农定理实际上是德·摩根定理的推广,它可以用在任何复杂函数。

【例 3.1】 已知函数 $F = \bar{A}B + A\bar{B}(C + \bar{D})$, 求其反函数 $\bar{F}$ 。

$$\begin{aligned}\bar{F} &= \overline{\bar{A}B + A\bar{B}(C + \bar{D})} = \overline{\bar{A}B} \cdot \overline{A\bar{B}(C + \bar{D})} \\ &= (A + \bar{B}) \cdot (\overline{A\bar{B}} + \overline{C + \bar{D}}) = (A + \bar{B})(\bar{A} + B + \bar{C}\bar{D})\end{aligned}$$

利用香农定理,可以直接写出

$$\bar{F} = (A + \bar{B}) \cdot (\bar{A} + B + \bar{C}\bar{D})$$

定理 3 展开定理。

$$(1) f(x_1, x_2, \dots, x_i, \dots, x_n)$$

$$= x_i f(x_1, x_2, \dots, 1, \dots, x_n) + \bar{x}_i f(x_1, x_2, \dots, 0, \dots, x_n)$$

$$(2) f(x_1, x_2, \dots, x_i, \dots, x_n)$$

$$= (x_i + f(x_1, x_2, \dots, 0, \dots, x_n)) \cdot (\bar{x}_i + f(x_1, x_2, \dots, 1, \dots, x_n))$$

这就是说,任何布尔函数都可以对它的某一变量 x_i 展开,或展开成(1)所示的“与-或”形式,或展开为(2)所示的“或-与”形式。

证明 将 $x_i = 1, \bar{x}_i = 0$ 代入上式,再将 $x_i = 0, \bar{x}_i = 1$ 代入上式,则两种情况下等式均成立。证毕。

由展开定理可得下列两个推理。

推理 1

$$(a) x_i f(x_1, \dots, x_i, \dots, x_n) = x_i f(x_1, \dots, 1, \dots, x_n)$$

$$(b) x_i + f(x_1, \dots, x_i, \dots, x_n) = x_i + f(x_1, \dots, 0, \dots, x_n)$$

推理 2

$$(a) \bar{x}_i f(x_1, \dots, x_i, \dots, x_n) = \bar{x}_i f(x_1, \dots, 0, \dots, x_n)$$

$$(b) \bar{x}_i + f(x_1, \dots, x_i, \dots, x_n) = \bar{x}_i + f(x_1, \dots, 1, \dots, x_n)$$

下面举例说明展开定理的应用。

【例 3.2】 证明公式 $AB + \bar{A}C + BC = AB + \bar{A}C$ (包含律)。

用展开定理,等号左边可展开成

$$\begin{aligned}\text{左边} &= A(1 \cdot B + 0 \cdot C + BC) + \bar{A}(0 \cdot B + 1 \cdot C + BC) \\ &= A(B + BC) + \bar{A}(C + BC) = AB + \bar{A}C = \text{右边}\end{aligned}$$

证毕

【例 3.3】 将函数 $F = \bar{A}\bar{B} + AC$ 表示成“或-与”形式。

由展开定理,可得

$$F = [A + (1 \cdot \bar{B} + 0 \cdot C)] \cdot [\bar{A} + (0 \cdot \bar{B} + 1 \cdot C)] = (A + \bar{B})(\bar{A} + C)$$

3.2.3 布尔代数的重要规则

布尔代数有 3 个重要规则,即代入规则、反演规则和对偶规则,现分别叙述如下。

1. 代入规则

任何一个含有变量 x 的等式,如果将所有出现 x 的位置,都代之以一个布尔函数 F ,则等式仍然成立。这个规则称为代入规则。

由于任何一个布尔函数也和任何一个变量一样,只有 0 或 1 两种取值,显然,以上规则是成立的。

【例 3.4】 已知等式 $\overline{A+B} = \overline{A} \cdot \overline{B}$, 函数 $F = B + C$, 若用 F 代入此等式中的 B , 则有

$$\overline{A + (B + C)} = \overline{A} \cdot \overline{B + C}$$

$$\overline{A + B + C} = \overline{A} \cdot \overline{B} \cdot \overline{C}$$

据此可以证明 n 变量的德·摩根定理的成立。

2. 对偶规则

任何一个布尔函数表达式 F , 如果将表达式中的所有的“+”改成“·”, “·”改成“+”, “1”改成“0”, “0”改成“1”, 而变量保持不变, 则可得到一个新的函数表达式 F_d , 我们称 F_d 为 F 的对偶函数, 这一规则称为对偶规则。例如, 下列为几个原函数及其对偶函数:

$$F = \overline{A}B + A\overline{B}C$$

$$F_d = (\overline{A} + B)(A + \overline{B} + \overline{C})$$

$$F = A(\overline{B} + CD) + E$$

$$F_d = [A + \overline{B}(C + D)] \cdot E$$

$$F = (A + 0) \cdot (B + C \cdot 1)$$

$$F_d = A \cdot 1 + B \cdot (C + 0)$$

$$F = A + B + \overline{\overline{C} + \overline{D} + \overline{E}}$$

$$F_d = A \cdot B \cdot \overline{\overline{C} \cdot \overline{D} \cdot \overline{E}}$$

需要注意的是,在运用对偶规则求对偶函数时,必须按照先“与”后“或”的顺序,否则容易写错,如 $F = \overline{A}B + A\overline{B}C$, 若求出对偶函数 $F_d = \overline{A} + B \cdot A + \overline{B} + \overline{C}$, 是错误的。因此,要特别注意原来函数中的“与”项,当这些“与”项变为“或”项时,应加括号。

从上面这些例子可以看出,如果 F 的对偶函数为 F_d , 则 F_d 的对偶函数就是 F 。也就是说, F 和 F_d 互为对偶函数, 即 $(F_d)_d = F$ 。

由布尔代数的基本公式可以看出,它们都是成对出现的互为对偶的等式。由此证明一个规则: 如果两个函数的表达式相等,则它们的对偶函数也相等,即如果函数 $F = G$, 则其对偶函数 $F_d = G_d$ 。

3. 反演规则

任何一个布尔函数表达式 F , 如果将表达式中的所有的“+”改成“·”, “·”改成“+”, “1”改成“0”, “0”改成“1”, 原变量改成反变量, 反变量改成原变量, 则可得函数 F 的反函数(或称补函数) $\overline{F}$, 这个规则称为反演规则。

实际上,反演规则就是香农定理。运用反演规则可以很方便地求一个函数的补函数。例如,下列为几个原函数及其补函数:

$$F = \overline{A}B + A\overline{B}C$$

$$\overline{F} = (A + \overline{B})(\overline{A} + B + C)$$

$$F = A(\overline{B} + CD) + E$$

$$\overline{F} = [\overline{A} + B(\overline{C} + \overline{D})] \cdot \overline{E}$$

$$F = (A + 0) \cdot (B + C \cdot 1)$$

$$\overline{F} = \overline{A} \cdot 1 + \overline{B} \cdot (\overline{C} + 0)$$

$$F = A + B + \overline{\overline{C} + \overline{D} + \overline{E}}$$

$$\overline{F} = \overline{A} \cdot \overline{B} \cdot \overline{C} \cdot \overline{D} \cdot \overline{E}$$

与求对偶函数一样,求补函数需要注意的是,在运用反演规则时,必须按照先“与”后“或”的顺序进行变换。因此,特别要注意原来函数中的“与”项,当这些“与”项变换为“或”项时,应加括号。

把上述补函数的例子与前面对偶函数的例子对照一下,可以看出,补函数和对偶函数之间在形式上只差变量的“非”。因此,若已求得一函数的对偶函数,只要将所有变量取反便得该函数的补函数;反之亦然。

3.3 布尔函数的基本形式

一个布尔函数的表达式可以有多种表示形式,本节讨论几种基本形式,它们是进行布尔函数化简的基础。

3.3.1 函数的“积之和”与“和之积”形式

根据展开定理,任何一个 n 变量函数总可以展开成“与-或”形式,或者“或-与”形式。其中“与-或”形式又称为“积之和”形式,而“或-与”形式又称为“和之积”形式。

所谓“积之和”,是指一个函数表达式中包含若干个“积”项,其中每个“积”项可有一个或多个以原变量或反变量形式出现的字母,这些“积”项的“和”就表示了一个函数。例如,一个三变量函数为

$$F(A, B, C) = \bar{A} + B\bar{C} + A\bar{B}C$$

其中, $\bar{A}$ 、 $B\bar{C}$ 、 $A\bar{B}C$ 均为“积”项。这些积项的和,就表示了函数的“积之和”形式。

所谓“和之积”,是指一个函数表达式中包含若干个“和”项,其中每个“和”项可有一个或多个以原变量或反变量形式出现的字母,这些“和”项的“积”就表示一个函数。例如,一个四变量函数为

$$F(A, B, C, D) = (A + B)(C + \bar{D})(\bar{A} + B + C)$$

其中, $(A + B)$ 、 $(C + \bar{D})$ 、 $(\bar{A} + B + C)$ 均为“和”项,这些“和”项的“积”就构成了函数的“和之积”形式。

当然,布尔函数还可表示成其他形式。例如,上述四变量函数还可表示成

$$F(A, B, C, D) = (AC + B)(CD + \bar{D}) = (AC + B)CD + (AC + B)\bar{D}$$

这种表示形式既不是“积之和”形式,又不是“和之积”形式,但它可以转换成后两种形式。

一个函数可以有多种表示形式,那么能不能找到统一的表示形式呢? 有两种统一的标准形式。

3.3.2 函数的“标准积之和”与“标准和之积”形式

1. 标准积之和

所谓标准积,是指函数的积项包含了函数的全部变量,其中每个变量都以原变量或反变

量的形式出现,且仅出现一次。标准积项通常称为最小项。

一个函数可以用最小项之和的形式来表示,称为函数的“标准积之和”形式。例如,一个三变量函数为

$$F(A, B, C) = \bar{A}\bar{B}\bar{C} + \bar{A}BC + A\bar{B}\bar{C} + ABC$$

它由 4 个最小项组成,这是函数的“标准积之和”形式。

由最小项的定义可知,3 个变量最多可组成 8 个最小项: $\bar{A}\bar{B}\bar{C}$ 、 $\bar{A}\bar{B}C$ 、 $\bar{A}B\bar{C}$ 、 $\bar{A}BC$ 、 $A\bar{B}\bar{C}$ 、 $A\bar{B}C$ 、 $AB\bar{C}$ 、 ABC 。为了叙述和书写方便,通常用 m_i 表示最小项,其下标 i 是这样确定的:把最小项中原变量记为 1,反变量记为 0,当变量顺序确定后,可以按顺序排列成一个二进制数。那么,与这个二进制数相对应的十进制数就是最小项的下标 i 。表 3.6 列出了 3 个变量的全部最小项。

表 3.6 三变量的所有最小项和最大项

A B C	最小项	最大项
0 0 0	$m_0 = \bar{A}\bar{B}\bar{C}$	$M_0 = A+B+C$
0 0 1	$m_1 = \bar{A}\bar{B}C$	$M_1 = A+B+\bar{C}$
0 1 0	$m_2 = \bar{A}B\bar{C}$	$M_2 = A+\bar{B}+C$
0 1 1	$m_3 = \bar{A}BC$	$M_3 = A+\bar{B}+\bar{C}$
1 0 0	$m_4 = A\bar{B}\bar{C}$	$M_4 = \bar{A}+B+C$
1 0 1	$m_5 = A\bar{B}C$	$M_5 = \bar{A}+B+\bar{C}$
1 1 0	$m_6 = AB\bar{C}$	$M_6 = \bar{A}+\bar{B}+C$
1 1 1	$m_7 = ABC$	$M_7 = \bar{A}+\bar{B}+\bar{C}$

因此,上述函数 $F(A, B, C)$ 可以写成

$$F(A, B, C) = \bar{A}\bar{B}\bar{C} + \bar{A}BC + A\bar{B}\bar{C} + ABC = m_2 + m_3 + m_4 + m_7 = \sum m(2, 3, 4, 7)$$

其中,符号“ $\sum$ ”表示各最小项求“或”,括号内的十进制数字表示各最小项的下标。

最小项有下列 3 个主要性质。

- (1) 对于任意一个最小项,只有一组变量取值使其值为 1。
- (2) 任意两个不同的最小项之积必为 0,即

$$m_i \cdot m_j = 0 \quad (i \neq j)$$

- (3) n 变量的所有 2^n 个最小项之和必为 1,即

$$\sum_{i=0}^{2^n-1} m_i = 1$$

用展开定理可以证明,任一个 n 变量的函数都有一个且仅有一个最小项表达式,即“标准积之和”形式。下面介绍求函数的“标准积之和”的两种常用的方法。

方法一:代数演算法,即通过反复使用公式 $x+\bar{x}=1$ 和 $x(y+z)=xy+xz$ 而求得“标准积之和”的方法。例如,设 $F(A, B, C) = \bar{A}B + A\bar{B}\bar{C} + BC$,则得

$$\begin{aligned} F(A, B, C) &= \bar{A}B(C+\bar{C}) + A\bar{B}\bar{C} + BC(A+\bar{A}) \\ &= \bar{A}BC + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC + \bar{A}BC \end{aligned}$$

$$\begin{aligned}
 &= \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C} + A\overline{B}\overline{C} + ABC = m_2 + m_3 + m_4 + m_7 \\
 &= \sum m(2,3,4,7)
 \end{aligned}$$

方法二：列表法，即列出函数的真值表，使函数取值为1的那些最小项，就构成了函数的“标准积之和”形式。例如，函数 $F(A,B,C) = \overline{A}B + A\overline{B}\overline{C} + BC$ 的真值表列于表3.7。根据真值表可以很方便地写出函数的表达式为

$$F(A,B,C) = m_2 + m_3 + m_4 + m_7 = \sum m(2,3,4,7)$$

式中， m_2 、 m_3 、 m_4 和 m_7 是相应于真值表中使函数取值为1的那些最小项。

表3.7 函数的真值表与最小项

A	B	C	$F(A,B,C)$	最小项
0	0	0	0	
0	0	1	0	
0	1	0	1	$m_2 = \overline{A}B\overline{C}$
0	1	1	1	$m_3 = \overline{A}BC$
1	0	0	1	$m_4 = A\overline{B}\overline{C}$
1	0	1	0	
1	1	0	0	
1	1	1	1	$m_7 = ABC$

2. 标准和之积

所谓标准和，是指函数的和项包含了全部变量，其中每个变量都以原变量或反变量形式出现，且仅出现一次。标准和项通常又称为最大项。

一个函数可以用最大项之积的形式表示，我们把这种形式称为函数的“标准和之积”形式。例如，一个三变量函数为

$$F(A,B,C) = (A+B+C)(A+B+\overline{C})(\overline{A}+B+\overline{C})(\overline{A}+\overline{B}+C)$$

它由4个最大项组成，这就是函数的“标准和之积”形式。

同样，3个变量最多可组成8个最大项，如表3.6所示。通常，最大项用 M_i 来表示，其下标 i 是这样确定的：当最大项的各变量按一定次序排好后，把其中的原变量记为0，反变量记为1，便得到一个二进制数，与该二进制数相应的十进制数就是最大项的下标 i 。

这样，上述函数 $F(A,B,C)$ 可以写成

$$\begin{aligned}
 F(A,B,C) &= (A+B+C)(A+B+\overline{C})(\overline{A}+B+\overline{C})(\overline{A}+\overline{B}+C) \\
 &= M_0 M_1 M_5 M_6 = \prod M(0,1,5,6)
 \end{aligned}$$

其中，符号“ $\prod$ ”表示各最大项相“与”，括号内的十进制数表示各最大项的下标。

最大项具有下列3个主要性质：

(1) 对于任意一个最大项，只有一组变量取值可使其值为0。

(2) 任意两个不同的最大项之和必为1，即

$$M_i + M_j = 1 \quad (i \neq j)$$

(3) n 变量的所有 2^n 个最大项之积必为0，即

$$\prod_{i=0}^{2^n-1} M_i = 0$$

同样地用展开定理可以证明,任何 n 变量的函数都有一个且仅有一个最大项表达式,即“标准和之积”形式。求函数的“标准和之积”的方法也有两种方法:

方法一:代数演算法,即通过反复地使用公式 $x \cdot \bar{x} = 1$ 和 $x + yz = (x + y)(x + z)$ 而求得“标准和之积”的方法。例如

$$\begin{aligned} F &= \bar{A}B + A\bar{B}\bar{C} + BC = (\bar{A}B + A)(\bar{A}B + \bar{B})(\bar{A}B + \bar{C}) + BC \\ &= (\bar{A} + A)(B + A)(\bar{A} + \bar{B})(B + \bar{B})(\bar{A} + \bar{C})(B + \bar{C}) + BC \\ &= (A + B)(\bar{A} + \bar{B})(\bar{A} + \bar{C})(B + \bar{C}) + BC \\ &= ((A + B)(\bar{A} + \bar{B})(\bar{A} + \bar{C})(B + \bar{C}) + B)((A + B)(\bar{A} + \bar{B})(\bar{A} + \bar{C})(B + \bar{C}) + C) \\ &= ((A + B + B)(\bar{A} + \bar{B} + B)(\bar{A} + \bar{C} + B)(B + \bar{C} + B)) \cdot \\ &\quad ((A + B + C)(\bar{A} + \bar{B} + C)(\bar{A} + \bar{C} + C)(B + \bar{C} + C)) \\ &= (A + B)(\bar{A} + B + \bar{C})(B + \bar{C})(A + B + C)(\bar{A} + \bar{B} + C) \end{aligned}$$

由于表达式中第一项缺少变量 C ,所以要加上 $C \cdot \bar{C}$;第三项缺少变量 A ,所以要加 $A \cdot \bar{A}$,即

$$\begin{aligned} F &= (A + B + C \cdot \bar{C})(\bar{A} + B + \bar{C})(A \cdot \bar{A} + B + \bar{C})(A + B + C)(\bar{A} + \bar{B} + C) \\ &= (A + B + C)(A + B + \bar{C})(\bar{A} + B + \bar{C})(\bar{A} + \bar{B} + C) \\ &= M_0 M_1 M_5 M_6 = \prod M(0, 1, 5, 6) \end{aligned}$$

可见,用这种方法是比较麻烦的。

方法二:列表法,即列出函数的真值表,那些使函数取值为 0 的最大项,就构成了函数的“标准和之积”形式。例如,上述函数的真值表列于表 3.8,根据真值表可以很方便地写出表达式为

$$F(A, B, C) = M_0 M_1 M_5 M_6 = \prod M(0, 1, 5, 6)$$

表 3.8 函数的真值表与最大项

A B C	$F(A, B, C)$	最大项
0 0 0	0	$M_0 = A + B + C$
0 0 1	0	$M_1 = A + B + \bar{C}$
0 1 0	1	
0 1 1	1	
1 0 0	1	
1 0 1	0	$M_5 = \bar{A} + B + \bar{C}$
1 1 0	0	$M_6 = \bar{A} + \bar{B} + C$
1 1 1	1	

比较表 3.7 和表 3.8,可以得出两点结论:

- (1) 同一个函数既可以表示成“标准积之和”的形式,又可表示成“标准和之积”的形式。

对于本例,有

$$F(A,B,C) = \bar{A}B + A\bar{B}C + BC = \sum m(2,3,4,7) = \prod M(0,1,5,6)$$

(2) 同一函数的最大项与最小项是互斥的,即如果真值表中的某一行作为函数的最小项,那么它就不可能是同一函数的最大项;反之亦然。一般有 $m_i = \overline{M_i}$ 或者 $M_i = \overline{m_i}$ 。换句话说,一个布尔函数的最小项的集合与它的最大项的集合,互为补集。因此,若已知一布尔函数的“标准积之和”形式,就可以很容易写出该函数的“标准和之积”形式。例如,已知函数的“标准积之和”形式为

$$F(A,B,C) = \sum m(1,3,4,6,7)$$

则该函数的“标准和之积”形式为

$$F(A,B,C) = \prod M(0,2,5)$$

3.4 不完全确定的布尔函数

前面所讨论的布尔函数都是属于完全确定的布尔函数。也就是说,它们的每一组输入变量的取值,都能得到一个完全确定的函数值(0 或 1)。如果布尔函数有 n 个变量,函数就有 2^n 个最小项(或最大项),其中每一项都有确定的值。

在实际应用中,有时只要求某些最小项(或最大项)有确定的值,而对其余最小项(或最大项)的取值不感兴趣,它们既可为 0,也可为 1,即随意取值。通常,把这种可以随意取值的最小项(或最大项)称为随意项,或无关项,记为 d 。这种具有随意项的布尔函数称为不完全确定的布尔函数。

在数字系统的设计中,这种不完全确定的布尔函数是经常遇到的。例如,如果逻辑电路的输入是二进制编码的十进制数,4 位二进制输入共有 16 种不同的状态,其中只有 10 种是允许的,有确定的输出;而其余 6 种是不允许的,因此它们的输出结果是人们不关心的,换句话说,结果可以是任意的。在设计中,可以充分利用这些任意项,使设计得到简化。

【例 3.5】 设计一个奇偶判别电路,其输入为一位十进制的 BCD 码。当输入为偶数时,电路输出为 0;当输入为奇数时,电路输出为 1,如图 3.4 所示。

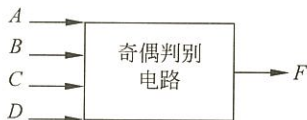


图 3.4 奇偶判别电路

根据设计要求可以列出描述该电路的布尔函数真值表,如表 3.9 所示。其中,第 10~15 行是不确定的,所以这是一个不完全确定的布尔函数,函数的表达式为:

$$F(A,B,C,D) = \sum m(1,3,5,7,9) + \sum d(10,11,12,13,14,15)$$

式中, d 表示随意项,可取 0,也可取 1。

表 3.9 BCD 码奇偶判别电路真值表

十进制数 r	BCD				$F(A, B, C, D)$
	A	B	C	D	
0	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	0
3	0	0	1	1	1
4	0	1	0	0	0
5	0	1	0	1	1
6	0	1	1	0	0
7	0	1	1	1	1
8	1	0	0	0	0
9	1	0	0	1	1
10	1	0	1	0	d
11	1	0	1	1	d
12	1	1	0	0	d
13	1	1	0	1	d
14	1	1	1	0	d
15	1	1	1	1	d

为了使设计的电路简单,可以将函数化简。下面分两种情况来考虑。

(1) 如果不考虑随意项(即取 $d=0$),则有

$$\begin{aligned}
 F(A, B, C, D) &= \sum m(1, 3, 5, 7, 9) \\
 &= \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}CD + \bar{A}B\bar{C}D + \bar{A}BCD + A\bar{B}\bar{C}D \\
 &= \bar{A}\bar{B}D(C + \bar{C}) + \bar{A}BD(C + \bar{C}) + \bar{B}\bar{C}D(\bar{A} + A) \\
 &= \bar{A}\bar{B}D + \bar{A}BD + \bar{B}\bar{C}D \\
 &= \bar{A}D + \bar{B}\bar{C}D
 \end{aligned} \tag{3.1}$$

(2) 如果考虑随意项,且取随意项 11、13、15 的值为 1,其余随意项取值为 0,则有

$$\begin{aligned}
 F(A, B, C, D) &= \sum m(1, 3, 5, 7, 9) + \sum_{d=1} d(11, 13, 15) \\
 &= \sum m(1, 3, 5, 7, 9, 11, 13, 15) = D
 \end{aligned} \tag{3.2}$$

式(3.1)比式(3.2)要简单得多。由此可见,对随意项合理赋值,可以使函数大为简化。因而其相应的逻辑电路也简单了。

3.5 布尔函数的化简

由 3.3 节讨论知道,同一个布尔函数可以有多种表示形式。一种形式的函数表达式相应于一种逻辑电路,尽管它们的形式不同,但其逻辑功能是相同的。函数表达式有简有繁,相应的逻辑电路也有简有繁,人们希望用尽可能少的逻辑门来完成同样的逻辑功能,这就要求函数表达式是最简单的。因此,如何使函数的表达式最简单,即函数的化简,成为逻辑设

计的一个关键问题。因为函数越简单,所设计的电路就不仅简单、经济,而且出现故障的可能性也越少,可靠性就越好。

在函数的各种不同形式的表达式中,“与或”表达式是最基本的,其他形式的表达式都可由它变换而得。例如

$$\begin{aligned}
 F(A, B, C) &= \overline{A}\overline{B} + AC && \text{ (“与或”形式)} \\
 &= (\overline{A} + C)(A + \overline{B}) && \text{ (“或与”形式)} \\
 &= \overline{\overline{A}\overline{B} \cdot AC} && \text{ (“与非”形式)} \\
 &= \overline{\overline{A} + C + \overline{A} + \overline{B}} && \text{ (“或非”形式)} \\
 &= \overline{AC} + \overline{AB} && \text{ “与或非”形式}
 \end{aligned}$$

因此,将从“与或”表达式出发来讨论函数的化简方法。

什么是函数的最简“与或”式呢? 一个最简“与或”式应同时满足以下两个条件。

(1) 该式中的与项最少。

(2) 该式中的每个与项的变量也最少。

这样,用逻辑门来实现布尔函数时,所需的与门数目最少,而且每个与门的输入端数目也最少。

本节讨论布尔函数的几种常用的化简方法,以及用不同门电路来实现布尔函数的方法。这些方法是组合逻辑网络设计的基础,应熟练掌握。

3.5.1 代数化简法

所谓代数化简法,就是运用布尔代数的基本公式、定理和规则来化简布尔函数的一种方法。这种方法没有固定的步骤可以遵循,主要是凭对布尔代数的公式、定理和规则的熟练运用程度。尽管如此,我们还是可以总结出一些适用于大多数情况的方法。这些方法的基本思想是对布尔函数进行等式变换,使表达式的与项减少,或使与项中的变量减少,以达到简化函数的目的。下面介绍化简“与或”表达式的几种常用的方法。

1. 并项法

利用公式 $AB + A\overline{B} = A$, 将两项合并为一项,并消去一个变量。例如

$$A\overline{B}C + A\overline{B}\overline{C} = A\overline{B}$$

$$A\overline{B}C + A\overline{B}\overline{C} = A$$

2. 吸收法

利用吸收律 $A + AB = A$, 消去多余的项。例如

$$\overline{B} + A\overline{B}D = \overline{B}$$

$$A\overline{B} + A\overline{B}CD(E + F) = A\overline{B}$$

还可以利用吸收律 $A + \overline{A}B = A + B$, 消去多余的变量。例如

$$\overline{A} + AB + DE = \overline{A} + B + DE$$

$$AB + \overline{A}C + \overline{B}C = AB + (\overline{A} + \overline{B})C = AB + \overline{ABC} = AB + C$$

3. 配项法

利用 $A \cdot 1 = A$ 和 $A + \bar{A} = 1$, 为某一项配上其所缺的一个变量, 以便用其他方法进行化简。例如

$$\begin{aligned} AB + \bar{A}C + BC &= AB + \bar{A}C + (A + \bar{A})BC = AB + \bar{A}C + ABC + \bar{A}BC \\ &= (AB + ABC) + (\bar{A}C + \bar{A}BC) = AB + \bar{A}C \quad (\text{吸收法}) \end{aligned}$$

还可以利用公式 $A + A = A$, 为某项配上其所能合并的项。例如

$$\begin{aligned} ABC + AB\bar{C} + A\bar{B}C + \bar{A}BC \\ &= (ABC + AB\bar{C}) + (ABC + A\bar{B}C) + (ABC + \bar{A}BC) \\ &= AB + AC + BC \end{aligned}$$

4. 消去冗余项法

等式 $AB + \bar{A}C + BC = AB + AC$ 可以作为一个基本公式使用, 它称为包含律, 其中 BC 称为冗余项。利用这个公式, 去寻找一个表达式的冗余项, 然后消去冗余项。例如

$$\begin{aligned} A\bar{B} + AC + ADE + \bar{C}D &= A\bar{B} + (AC + \bar{C}D + ADE) = A\bar{B} + AC + \bar{C}D \\ AB + \bar{B}C + AC(D + E) &= AB + \bar{B}C \end{aligned}$$

以上介绍了几种常用的方法。在实际应用中可能遇到比较复杂的函数, 只要熟练掌握布尔代数的公式和定理, 灵活运用上述方法, 总能找到化简的办法。下面是几个综合运用上述方法化简布尔函数的实例。

【例 3.6】 化简 $F = AD + A\bar{D} + AB + \bar{A}C + BD + ACEF + \bar{B}EF + DEFG$

$$\begin{aligned} F &= AD + A\bar{D} + AB + \bar{A}C + BD + ACEF + \bar{B}EF + DEFG \\ &= A + AB + \bar{A}C + BD + ACEF + \bar{B}EF + DEFG && (\text{并项法}) \\ &= A + \bar{A}C + BD + \bar{B}EF + DEFG && (\text{吸收法}) \\ &= A + C + BD + \bar{B}EF + DEFG && (\text{吸收法}) \\ &= A + C + BD + \bar{B}EF && (\text{消去冗余项}) \end{aligned}$$

【例 3.7】 化简 $F = A\bar{B} + B\bar{C} + \bar{B}C + \bar{A}B$

$$\begin{aligned} F &= A\bar{B} + B\bar{C} + \bar{B}C + \bar{A}B \\ &= A\bar{B} + B\bar{C} + (A + \bar{A})\bar{B}C + \bar{A}B(C + \bar{C}) && (\text{配项法}) \\ &= (A\bar{B} + A\bar{B}C) + (B\bar{C} + \bar{A}B\bar{C}) + (\bar{A}\bar{B}C + \bar{A}BC) \\ &= A\bar{B} + B\bar{C} + \bar{A}C && (\text{吸收法}) \end{aligned}$$

【例 3.8】 化简或与表达式

$$F = (\bar{B} + D)(\bar{B} + D + A + G)(C + E)(\bar{C} + G)(A + E + G)$$

可以利用对偶规则, 先求出 F 的对偶式:

$$F_d = \bar{B}D + \bar{B}DAG + CE + \bar{C}G + AEG$$

然后, 利用与或式的化简方法进行化简, 则得

$$F_d = \bar{B}D + CE + \bar{C}G$$

最后, 对 F_d 再求对偶式, 则得

$$F = (F_d)_d = (\bar{B} + D)(C + E)(\bar{C} + G)$$

由本例可见, 与或表达式的化简是基础, 其他形式的表达式都可先变换为与或表达式进

行化简,然后再变换成所需的形式。

从以上的例子可以看出,代数化简法不仅使用不便,而且难以判断所得之结果是否为最简。因此,代数化简法一般适用于函数表达式较为简单的情况。当函数较为复杂时,往往采用比较方便的、有规则的卡诺图法。

3.5.2 卡诺图化简法

卡诺图化简法是将布尔函数用卡诺图来表示,在卡诺图上进行函数的化简的方法。这是一种很简单、直观的方法。

1. 卡诺图的构成

卡诺图是真值表的图形化。我们可以把真值表看成是由最小项构成的一个纵列。一个函数由若干个最小项构成,则在相应的最小项上填1,其余填0。

例如,设有函数 $F(A,B,C) = \sum m(2,3,5,7)$, 该函数的真值表如表 3.10 所示。

该函数可以进行化简,即

$$\begin{aligned} F(A,B,C) &= \bar{A}\bar{B}\bar{C} + \bar{A}BC + A\bar{B}\bar{C} + ABC \\ &= \bar{A}B(\bar{C} + C) + AC(\bar{B} + B) \\ &= \bar{A}B + AC \end{aligned}$$

其中,最小项 $\bar{A}\bar{B}\bar{C}$ 和 $\bar{A}BC$ (或者 $A\bar{B}\bar{C}$ 和 ABC) 只有一个变量互补,其余变量相同,称这样的最小项为相邻最小项,它们可以合并消去一个变量。因此,化简的实质是两个相邻最小项的合并。

表 3.10 $F(A,B,C) = \sum m(2,3,5,7)$ 的真值表

A	B	C	$F(A,B,C)$	最小项
0	0	0	0	m_0
0	0	1	0	m_1
0	1	0	1	m_2
0	1	1	1	m_3
1	0	0	0	m_4
1	0	1	1	m_5
1	1	0	0	m_6
1	1	1	1	m_7

从真值表上很难直观地看出最小项的相邻关系的。例如,相邻最小项 m_5 和 m_7 在位置上并不相邻。那么,能不能把最小项排成一种图,可以直观地从图上看最小项的相邻关系呢? 可以的。从 Gray 码的特点知道,两个相邻代码之间只有一位不同。因此,只要把真值表中的最小项重新排列,把它们排列成矩阵形式,并且使矩阵的横方向和纵方向的布尔变量的取值按 Gray 码的顺序排列,这样构成的图形就是卡诺图。图 3.5 表示了二变量、三变量和四变量的卡诺图的构成。

从卡诺图可以看出,任意两个相邻的最小项在图上是相邻的。并且,图中最左列的最小

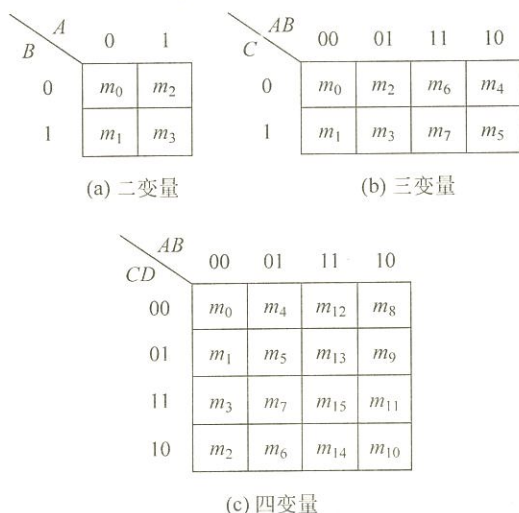


图 3.5 卡诺图的构成

项与最右列的相应最小项也是相邻的；位于最上面一行的最小项与最下面一行的相应最小项也是相邻的。因此，每个二变量的最小项有两个最小项与它相邻；每个三变量的最小项有 3 个最小项与它相邻；每个四变量的最小项有 4 个最小项与它相邻。可以证明每个 n 变量的最小项有 n 个最小项与它相邻。

五变量的卡诺图如图 3.6 所示。图中方格内的数字为最小项的下标。它可以看成是由两个四变量的卡诺图构成的，只要把右边部分的四变量卡诺图重合到左边部分的四变量卡诺图上来，就可找出各最小项的相邻关系。例如，最小项 3 与最小项 1、2、7、11 及 19 是相邻的。

		ABC							
		000	001	011	010	100	101	111	110
DE	00	0	4	12	8	16	20	28	24
	01	1	5	13	9	17	21	29	25
	11	3	7	15	11	19	23	31	27
	10	2	6	14	10	18	22	30	26

图 3.6 五变量卡诺图

六变量卡诺图如图 3.7 所示，它可以看成是由 4 个四变量卡诺图构成的。在寻找各最小项的相邻关系时，除了要注意左、右两个四变量卡诺图的重合关系外，还要注意上、下两个四变量卡诺图的重合关系。例如，最小项 3 除与最小项 1、2、7、11、19 相邻外，还与最小项 35 相邻。

变量多于 6 个时，卡诺图就显得很庞大，在实际应用中已失去它的优越性，一般就很少用它了。

2. 布尔函数在卡诺图上的表示

从卡诺图的构成方法可知，卡诺图实际是真值表的重新排列，使最小项排列得更紧凑、

ABC DEF		000 001 011 010				100 101 111 110			
		000	001	011	010	100	101	111	110
000		0	4	12	8	16	20	28	24
001		1	5	13	9	17	21	29	25
011		3	7	15	11	19	23	31	27
010		2	6	14	10	18	22	30	26
100		32	36	44	40	48	52	60	56
101		33	37	45	41	49	53	61	57
111		35	39	47	43	51	55	63	59
110		34	38	46	42	50	54	62	58

图 3.7 六变量卡诺图

更便于化简。因此,根据卡诺图与真值表的对应关系,就可以知道布尔函数在卡诺图上的表示方法。

如果布尔函数是以真值表的形式或者以“标准积之和”的形式给出的,我们只要在卡诺图上找出那些与给定布尔函数的最小项相对应的方格,并标以 1,就得到该函数的卡诺图。例如,三变量函数

$$F(A,B,C) = \sum m(2,3,5,7)$$

其卡诺图如图 3.8 所示。

如果布尔函数是“与或”表达式,则要将各“与项”分别标在卡诺图上。例如,给定函数的表达式为

$$F(A,B,C) = AB + A\bar{C}$$

只要在 AB 为 11 的列标以 1,并在 A 为 1、 C 为 0 的对应方格中标上 1,便可得到如图 3.9 所示的卡诺图。

AB C		00	01	11	10
		0	1	0	0
0		0	1	0	0
1		0	1	1	1

图 3.8 $F(A,B,C) = \sum m(2,3,5,7)$ 的卡诺图

$\begin{array}{c} AB \\ C \end{array}$		00	01	11	10
		0	0	1	1
C	0	0	0	1	1
	1	0	0	1	0

图 3.9 $F(A,B,C) = AB + A\bar{C}$ 的卡诺图

AB CD		00	01	11	10
		00	01	11	10
00		1	0	0	1
01		1	1	0	0
11		1	1	0	0
10		1	0	0	1

图 3.10 $F(A,B,C,D) = \bar{A}BD + \bar{B}\bar{D} + \bar{A}\bar{B}D$ 的卡诺图

又如,给定四变量函数为

$$F(A,B,C,D) = \bar{A}BD + \bar{B}\bar{D} + \bar{A}\bar{B}D$$

其卡诺图如图 3.10 所示。

3. 卡诺图的性质

卡诺图化简布尔函数的基本原理是基于卡诺图的性质。前面已指出,化简的实质是相邻最小项的合并。卡诺图的一个明显优点是它能利用人的直观的阅图能力,方便

地表示出所有相邻最小项。卡诺图具有下列性质。

性质 1 卡诺图上任何两个(2^1 个)标 1 的相邻最小项,可以合并为一项,并消去一个变量。

例如,在图 3.10 中,最小项 $m_1 = \bar{A}\bar{B}\bar{C}D$ 和 $m_3 = \bar{A}\bar{B}CD$ 相邻,所以它们可以合并;最小项 $m_5 = \bar{A}B\bar{C}D$ 和 $m_7 = \bar{A}BCD$ 相邻,所以它们也可以合并。即有

$$\bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}CD = \bar{A}\bar{B}D, \quad \bar{A}B\bar{C}D + \bar{A}BCD = \bar{A}BD$$

性质 2 卡诺图上任何 4 个(2^2 个)标 1 的相邻最小项,可以合并为一项,并消去两个变量。

例如,在图 3.10 中,最小项 m_1, m_3, m_5 和 m_7 彼此相邻,这 4 个最小项可以合并。即有

$$(m_1 + m_3) + (m_5 + m_7) = \bar{A}\bar{B}D + \bar{A}BD = \bar{A}D$$

这种合并,在卡诺图中表示为把 4 个 1 圈在一起。

在四变量卡诺图中,4 个标 1 的小方格相邻的典型情况如图 3.11 所示。

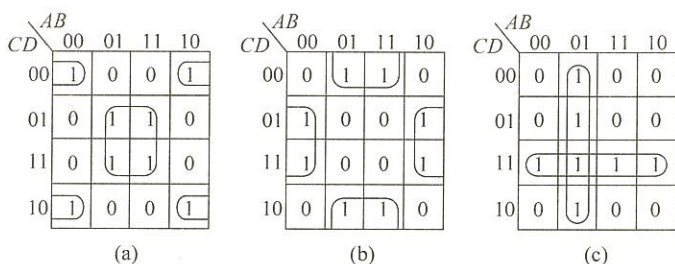


图 3.11 4 个相邻最小项合并的情况

在图 3.11(a)中,最小项 m_0, m_2, m_8, m_{10} 相邻,而最小项 m_5, m_7, m_{12}, m_{15} 也相邻。它们合并后,可得函数表达式为

$$F(A, B, C, D) = \bar{B}\bar{D} + BD$$

根据同样道理,将图 3.11(b)中的相邻最小项合并后,可得函数表达式为

$$F(A, B, C, D) = B\bar{D} + \bar{B}D$$

将图 3.11(c)中的相邻最小项合并后,结果为

$$F(A, B, C, D) = \bar{A}B + CD$$

性质 3 任何 8 个(2^3 个)标 1 的相邻最小项,可合并为一项,并消去 3 个变量。

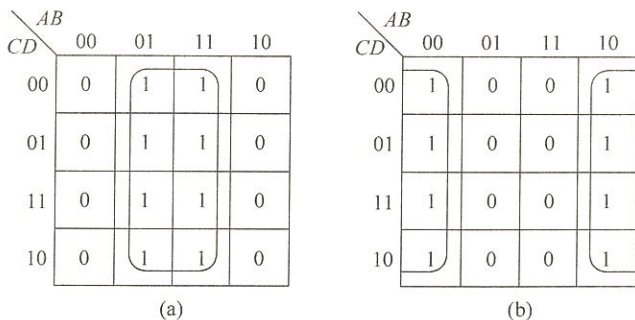


图 3.12 8 个相邻最小项合并的情况

图 3.12 表示了 8 个相邻最小项合并的情况。其中,图 3.12(a)所示的 8 个最小项合并后,结果为

$$F(A,B,C,D) = B$$

图 3.12(b)中的 8 个最小项合并后,结果为

$$F(A,B,C,D) = \bar{B}$$

由上述性质可知,相邻最小项的数目必须为 2^i 个才能合并成一项,并消去 i 个变量。包含的最小项数目越多,消去的变量也越多,从而所得到的逻辑表达式就越简单。

4. 卡诺图化简的基本步骤

在讨论用卡诺图化简的具体方法之前,先定义几个概念。

蕴涵项 在函数的任何与或表达式中,每个与项称为该函数的蕴涵项(Implicant)。显然,在函数的卡诺图中,任一标 1 的最小项以及由 2^i 个相邻最小项所形成的圈都是函数的蕴涵项。

质蕴涵项 如果函数的某一蕴涵项不是该函数中其他蕴涵项的一个子集,则此蕴涵项称为质蕴涵项(Prime Implicant)。从卡诺图上看,所谓质蕴涵项,就是大得不能再大的圈。例如,在图 3.13 中, BD 、 $\bar{A}\bar{C}\bar{D}$ 和 ABC 都是质蕴涵项,而 BCD 、 $\bar{A}BCD$ 等都不是质蕴涵项。

必要质蕴涵项 如果函数的一个质蕴涵项,至少包含了一个其他任何质蕴涵项都不包含的标 1 最小项,则此质蕴涵项称为必要质蕴涵项(Essential Prime Implicant)。例如,在图 3.13 中, BD 、 $\bar{A}\bar{C}\bar{D}$ 是必要质蕴涵项,而 ABC 就不是必要质蕴涵项。

根据以上定义,可以给出用卡诺图化简布尔函数的基本步骤如下。

- (1) 将布尔函数正确地标到卡诺图上,并在图上找出所有质蕴涵项。
- (2) 求出所有必要质蕴涵项。
- (3) 求函数的最小覆盖(即函数的最简表达式)。

下面通过具体例子来说明上述步骤。

【例 3.9】 用卡诺图法化简函数

$$F(A,B,C,D) = \sum m(0,3,4,5,7,11,13,15)$$

(1) 作 F 的卡诺图,并求得所有质蕴涵项为 BD 、 CD 、 $\bar{A}\bar{C}\bar{D}$ 、 $\bar{A}BC$,如图 3.14 所示。

(2) 求出所有必要质蕴涵项为 CD 、 BD 、 $\bar{A}\bar{C}\bar{D}$ 。

(3) 由于必要质蕴涵项的集合已覆盖了函数的所有最小项,因此函数的最简与或式为

$$F(A,B,C,D) = BD + CD + \bar{A}\bar{C}\bar{D}$$

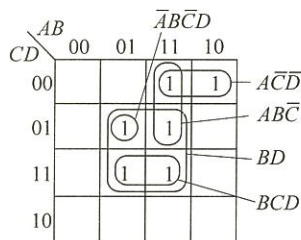


图 3.13 卡诺图中的质蕴涵项

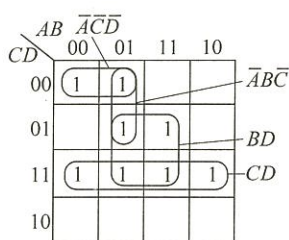


图 3.14 例 3.9 的卡诺图

【例 3.10】 用卡诺图化简布尔函数

$$F(A, B, C, D) = \sum m(0, 5, 6, 8, 15) + \sum d(1, 2, 3, 7, 10, 12, 13)$$

这是含有随意项 d 的情况。利用随意项求质蕴涵时, 如果它对化简有利, 则取 $d=1$; 如果它对化简不利, 则取 $d=0$ 。

(1) 作函数的卡诺图, 求出所有质蕴涵项为 $\overline{A}\overline{B}$ 、 $\overline{A}D$ 、 $\overline{A}C$ 、 BD 、 $\overline{B}\overline{D}$ 。如图 3.15 所示。

(2) 求出所有必要质蕴涵项为 $\overline{A}C$ 、 BD 、 $\overline{B}\overline{D}$ 。注意, 在求必要质蕴涵项时, 只要考虑 1 的覆盖。

(3) 求函数的最小覆盖。本例中必要质蕴涵已覆盖了卡诺图中所有标 1 的最小项。因此, 布尔函数的最简与或式为

$$F(A, B, C, D) = \overline{A}C + BD + \overline{B}\overline{D}$$

【例 3.11】 用卡诺图化简布尔函数

$$F(A, B, C, D) = \sum m(0, 4, 6, 7, 8, 9, 11, 12, 13, 15)$$

(1) 求出所有质蕴涵项为 $\overline{C}\overline{D}$ 、 $\overline{A}\overline{C}$ 、 AD 、 $\overline{A}B\overline{D}$ 、 $\overline{A}BC$ 、 BCD , 如图 3.16 所示。

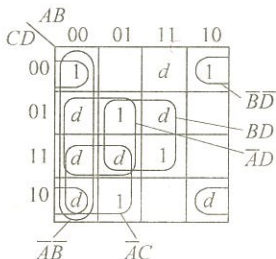


图 3.15 例 3.10 的卡诺图

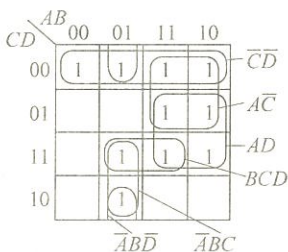


图 3.16 例 3.11 的卡诺图

(2) 求出所有必要质蕴涵项为 $\overline{C}\overline{D}$ 、 AD 。

(3) 求函数的最小覆盖。本例中必要质蕴涵只覆盖了函数的 8 个最小项, 还剩下两个最小项 m_6 、 m_7 未被覆盖。由观察可知, 在几个非必要质蕴涵中, 选择 $\overline{A}BC$ 这一项即可覆盖 m_6 、 m_7 。因此, 该函数的最简与或式为

$$F(A, B, C, D) = AD + \overline{C}\overline{D} + \overline{A}BC$$

这说明, 在一个最简与或式中, 并非每个质蕴涵项都是必要的, 其中可能包含所选择的质蕴涵项。这一点应引起注意。

必须指出, 由于卡诺图化简法带有试凑性质, 因此, 当读者已对卡诺图应用自如时, 就不必按上述步骤去做, 可以在卡诺图上一次画出最小覆盖。

【例 3.12】 用卡诺图化简函数

$$F(A, B, C, D) = \sum m(0, 1, 3, 4, 7, 12, 13, 15)$$

作出 F 的卡诺图如图 3.17(a) 所示。该函数有 8 个质蕴涵项, 它们相互交连, 找不出哪个是必要质蕴涵项, 这种情况通常称为循环结构。对于这类循环结构, 通常可选取一个最大的质蕴涵圈作为必要质蕴涵, 以打破循环结构。本例中, 由于各个质蕴涵圈的大小相等, 故可任选一个质蕴涵作为必要质蕴涵项。图 3.17(b) 为其中的一种解, 可得 F 的最简表达式为

$$F(A,B,C,D) = \overline{A}\overline{B}\overline{C} + \overline{A}CD + ABD + B\overline{C}\overline{D}$$

同理,可得另一个最简表达式为

$$F(A,B,C,D) = \overline{A}\overline{B}D + BCD + ABC\overline{C} + \overline{A}\overline{C}D$$

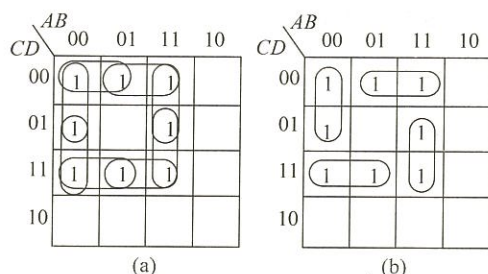


图 3.17 例 3.12 的卡诺图

【例 3.13】 化简五变量函数

$$F(A,B,C,D,E) = \sum m(2,4,5,6,7,12,13,18,20,21,22,23,24,25,28,29)$$

五变量布尔函数可用两个四变量卡诺图来表示,相重叠的最小项是相邻的,可以合并。画出函数 F 的卡诺图如图 3.18 所示。重叠部分的质蕴涵为 $\overline{B}D\overline{E}$ 、 $\overline{B}C$ 和 $\overline{C}D$; 不重叠的质蕴涵为 $AB\overline{D}$ 。这几个都为必要质蕴涵,且覆盖了函数的全部标 1 最小项。因此,函数的最简表达式为

$$F(A,B,C,D,E) = \overline{B}D\overline{E} + \overline{B}C + \overline{C}D + AB\overline{D}$$

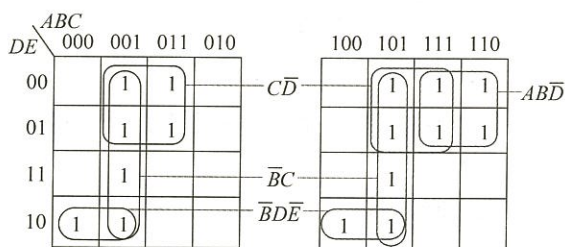


图 3.18 例 3.13 的卡诺图

由上述例子可见,用卡诺图化简布尔函数,简单明了,形象直观,容易掌握。

3.5.3 列表化简法

上节介绍的卡诺图化简法是手算常用的方法,它的缺点是:由于要靠人对图形的识别能力,因此不便于机器实现;而且,当函数的变量增多时(如多于 6 个),这种方法就逐渐失去它的优越性。

本节介绍一种更有规律的、系统的方法,即列表化简法。这一方法是由 W. V. Quine 和 E. J. McCluskey 在 1956 年研究发表的,故也称奎恩-麦克拉斯基法,简称 Q-M 法。这种方法的基本步骤与卡诺图法是相同的,即先找出布尔函数的所有质蕴涵,然后找出其中的必要质蕴涵,最后求函数的最小覆盖。所不同的是,列表化简法完成上述步骤是通过约定形式的表格,按照一定规则求得的。下面分别讨论列表法的这几个步骤。

1. 用列表法确定布尔函数的所有质蕴涵项

用列表法找布尔函数的所有质蕴涵项的基本思想是这样的：首先将布尔函数用最小项来表示，并把最小项表示成二进制数形式。例如

$m_4: \bar{A}\bar{B}\bar{C}\bar{D}$ 表示成 0100。

$m_5: \bar{A}\bar{B}\bar{C}D$ 表示成 0101。

然后进行两两比较，如果两个二进制数只有一位不同，就可合并。例如

$$\begin{array}{r} 0100 \\ > \\ 0101 \end{array} \text{ 合并为 } 010-$$

其中，“-”表示该位置上的变量被消去，同理，对于带有“-”的两个二进制数，若只有一位不相同(0 与 1)，则此二进制数又可进一步合并，得到带有两个“-”的二进制数。例如：

$$\begin{array}{r} 0100 \\ > \\ 0101 \\ > \\ 0110 \\ > \\ 0111 \end{array} \begin{array}{l} 010- \\ \\ 011- \end{array} \text{ 合并为 } 01--$$

就这样逐次合并只有一位数值不同的两个二进制数，所得到的不能再合并的二进制数，其对应的乘积项，即为质蕴涵项。

下面举例说明用列表法求质蕴涵的过程。

【例 3.14】 求下列函数的全部质蕴涵项：

$$F(A, B, C, D) = \sum m(1, 3, 4, 5, 10, 11, 12, 13)$$

为便于对照，以弄清合并的原理，不妨先给出卡诺图上合并的情况，如图 3.19 所示。

用 Q-M 法求质蕴涵过程如下：

第一步，列出最小项的二进制数形式，如表 3.11 所示。

表 3.11 最小项的二进制形式

m_i	A	B	C	D
1	0	0	0	1
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1

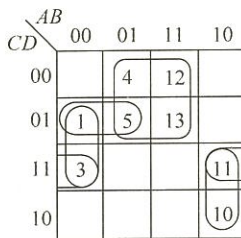


图 3.19 例 3.14 的卡诺图

可以看出，两个二进制数可以合并有如下规律：只有二进制数中 1 的个数相差 1 的两数才能合并，而 1 的个数相同或相差 2 和 2 以上的两数不能合并。这一规律启示我们，如果把上述二进制数按 1 的个数分组，那么，只有在相邻组之间才有合并的可能性，组内和隔组之间则不必考虑合并可能。这样可以大大地减少合并的工作量。

第二步,按1的个数分组列表,并在相邻组之间进行搜索合并。凡是能合并的两个项,则在其后面打“√”,表示它们不是质蕴涵项,并将合并结果列于另一栏中。如此继续,直到无法合并为止。最后,那些没有打“√”的项,就是质蕴涵项 P_i 。以上过程如表 3.12 所示。

最后,求得全部质蕴涵项为

$$P_1 = B\bar{C}, \quad P_2 = \bar{A}\bar{B}D, \quad P_3 = \bar{A}\bar{C}D, \quad P_4 = \bar{B}CD, \quad P_5 = A\bar{B}C$$

该结果与图 3.19 卡诺图上合并结果相同。

表 3.12 求质蕴涵项 P_i

m_i	A	B	C	D		m_i	A	B	C	D		m_i	A	B	C	D	
1	0	0	0	1	√	1,3	0	0	—	1	P_2	4,5	—	1	0	—	P_1
4	0	1	0	0	√	1,5	0	—	0	1	P_3	12,13	—	1	0	—	
3	0	0	1	1	√	4,5	0	1	0	—	√						
5	0	1	0	1	√	4,12	—	1	0	0	√						
10	1	0	1	0	√	3,11	—	0	1	1	P_4						
12	1	1	0	0	√	5,13	—	1	0	1	√						
11	1	0	1	1	√	10,11	1	0	1	—	P_5						
13	1	1	0	1	√	12,13	1	1	0	—	√						

2. 用质蕴涵表确定必要质蕴涵

所谓质蕴涵表,就是函数的各个质蕴涵覆盖函数的相应最小项的情况表。下面通过例子来说明如何列质蕴涵表,以及如何用质蕴涵表来确定必要质蕴涵。

【例 3.15】 已知函数

$$F(A,B,C,D) = \sum m(1,3,4,5,10,11,12,13)$$

求得它的全部质蕴涵为 $B\bar{C}$ 、 $\bar{A}\bar{B}D$ 、 $\bar{A}\bar{C}D$ 、 $\bar{B}CD$ 、 $A\bar{B}D$ 。列质蕴涵表如表 3.13 所示。表中,各纵列代表函数所包含的最小项,各横行是函数的质蕴涵项。每一个质蕴涵项所覆盖的最小项,都在表中行列交叉处用记号“×”标出,例如,质蕴涵项 $B\bar{C}$ 可覆盖最小项 4、5、12、13。因此,在 $B\bar{C}$ 这一行的最小项 4、5、12、13 列下标上“×”号。其他各行按同样方法标上记号。

表 3.13 质蕴涵表

$m_i \backslash P_j$	1	3	4	5	10	11	12	13
$\sqrt{B\bar{C}}$			⊗	×			⊗	⊗
$\bar{A}\bar{B}D$	×	×						
$\bar{A}\bar{C}D$	×			×				
$\bar{B}CD$		×				×		
$\sqrt{A\bar{B}D}$					⊗	×		
覆盖情况			×	×	×	×	×	×

必要质蕴涵是按下列步骤求得的:

(1) 逐列检查表 3.13 中标有“×”号的情况,凡只标有一个“×”号的列,则在该“×”的

外面打一个圈,即 $\otimes$ 。例如,表中最小项 4、10、12、13 各列都只有一个“ $\times$ ”,故都加上圈。

(2) 找出包含有 $\otimes$ 号的各行,这些行的质蕴涵项就是必要质蕴涵项,并在其前加上标记“ $\checkmark$ ”。例如,表中 $B\bar{C}$ 和 $A\bar{B}D$ 就为必要质蕴涵项。

(3) 在表的最后一行覆盖情况一栏中,标上必要质蕴涵项覆盖最小项的情况。凡能被必要质蕴涵项覆盖的最小项,在最后一行的该列上打“ $\times$ ”号。

由表 3.13 可知,本例中必要质蕴涵项 $B\bar{C}$ 和 $A\bar{B}D$,没有覆盖 F 的全部最小项。因此,接着要做的下一步是求 F 的最小覆盖。

3. 求函数的最小覆盖

一般来说,必要质蕴涵覆盖函数的最小项的情况有两种可能:一种是必要质蕴涵已覆盖了函数的所有最小项,对于这种情况,函数的简化工作就已完成,即函数的最小覆盖就是所有必要质蕴涵项。另一种是必要质蕴涵没有覆盖函数的所有最小项,对于这种情况,还需要选择适当的质蕴涵项,以覆盖剩下的最小项。那么,覆盖剩余的最小项所需的质蕴涵应该怎样选择才能使函数表达式最简呢?下面介绍两种常用的选取所需质蕴涵的方法。

1) 行列消去法

在质蕴涵表中,去掉必要质蕴涵项和已被其覆盖的最小项,剩下的部分称为简化的质蕴涵表。下面的两个规则——优势行规则和优势列规则,能从简化的质蕴涵表中选取所需的质蕴涵。

① 优势行规则。设质蕴涵 P_i 和 P_j 是简化质蕴涵表中的两行,其中 P_j 行中的“ $\times$ ”完全包含在 P_i 行中,则称 P_i 为优势行, P_j 为劣势行,记作 $P_i \supset P_j$ 。这时,在简化质蕴涵表中可以消去劣势行 P_j (这是因为选取了优势行 P_i 后,不仅可覆盖劣势行 P_j 所覆盖的最小项,而且还可覆盖其他最小项)。

例如,由表 3.13 去掉必要质蕴涵项和被它所覆盖的最小项,得到简化的质蕴涵表如表 3.14 所示。表中, P_3 相对于 P_2 来说是劣势行,即 $P_2 \supset P_3$,可消去 P_3 ; 同理, P_4 相对于 P_2 来说也是劣势行,即 $P_2 \supset P_4$,可消去 P_4 。最后只剩下 P_2 。

表 3.14 简化的质蕴涵表

$P_j \backslash m_i$	1	3
P_2	$\times$	$\times$
P_3	$\times$	
P_4		$\times$

表 3.15 优势列举例

$P_j \backslash m_i$	m_1	m_2	m_3
P_1	$\times$	$\times$	
P_2	$\times$		$\times$
P_3	$\times$		

② 优势列规则。设最小项 m_i 和 m_j 是简化质蕴涵表中的两列,其中 m_j 列中的“ $\times$ ”完全包含在 m_i 列之中,则称 m_i 为优势列, m_j 为劣势列,记作 $m_i \supset m_j$ 。这时,在简化质蕴涵表中可以消去优势列 m_i (这是因为选取了覆盖劣势列的质蕴涵后,一定能覆盖优势列,反之则不一定)。

例如,设表 3.15 为简化质蕴涵表,表中的 m_2 和 m_3 相对于 m_1 来说是劣势列,即 $m_2 \subset m_1, m_3 \subset m_1$,故可消去优势列 m_1 。

这种优势行规则和优势列规则可以反复交替使用,使简化的质蕴涵表进一步简化,以便

求得所需的质蕴涵项。

【例 3.16】 用行列消去法求所需质蕴涵项,设简化的质蕴涵表如表 3.16 所示。

表中,根据优势行规则,有 $P_4 \supset P_5, P_3 \supset P_6$,故可消去 P_5 和 P_6 ; 又根据优势列规则,由于 $m_4 \subset m_6, m_{10} \subset m_2$,故可消去 m_2 和 m_6 。最后,求得所需质蕴涵为 P_3 和 P_4 ,它们覆盖了简化质蕴涵表的所有最小项 m_2, m_4, m_6 和 m_{10} 。

表 3.16 简化的质蕴涵表

$m_i \backslash P_j$	m_2	m_4	m_6	m_{10}
P_2	×		×	
P_3	×			×
P_4		×	×	
P_5		×		
P_6				×

2) 布尔代数法

所谓布尔代数法,就是从简化质蕴涵表列出布尔表达式,从中选出最简的所需质蕴涵项。

例如,对于表 3.14 的简化质蕴涵表,要覆盖最小项 m_1 ,可选取质蕴涵项 P_2 或 P_3 ; 要覆盖最小项 m_3 ,可选取质蕴涵项 P_2 或 P_4 。因此,若要同时覆盖 m_1 和 m_3 ,可以用如下布尔表达式来表示:

$$(P_2 + P_3)(P_2 + P_4) = P_2 + P_2 P_4 + P_2 P_3 + P_3 P_4 = P_2 + P_3 P_4$$

该式表明,同时覆盖 m_1 和 m_3 的方案有两个,即选用 P_2 或选用 $P_3 P_4$,其中选用 P_2 最简单;该结果与行列消去法所得结果相同。

又如,对于表 3.16 的简化质蕴涵表,若要同时覆盖最小项 m_2, m_4, m_6 和 m_{10} ,则有

$$\begin{aligned} (P_2 + P_3)(P_4 + P_5)(P_2 + P_4)(P_3 + P_6) &= (P_3 + P_2 P_6)(P_4 + P_2 P_5) \\ &= P_3 P_4 + P_2 P_3 P_5 + P_2 P_4 P_6 + P_2 P_5 P_6 \end{aligned}$$

该式表明,可供选取的所需质蕴涵集有 4 个,其中只有 $P_3 P_4$ 最简单。所以,用布尔代数法求得所需质蕴涵项为 P_3 和 P_4 ,其结果与行列消去法所得结果相同。

最后,我们举两个综合性的例子,将前面所讲的连贯起来,以说明用 Q-M 列表法化简一个布尔函数的全过程。

【例 3.17】 化简布尔函数

$$F(A, B, C, D) = \sum m(2, 4, 6, 8, 9, 10, 12, 13, 15)$$

① 求函数的所有质蕴涵,如表 3.17 所示。

求得所有质蕴涵为

$$P_1 = AC, P_2 = \bar{A}\bar{C}\bar{D}, P_3 = \bar{B}\bar{C}\bar{D}, P_4 = \bar{A}B\bar{D}, P_5 = B\bar{C}\bar{D}, P_6 = A\bar{B}\bar{D}, P_7 = ABD$$

② 求必要质蕴涵,如表 3.18 所示。

由表 3.18 可确定 P_1 和 P_7 为必要质蕴涵。

表 3.17 求质蕴涵项 P_i

m_i	A	B	C	D		m_i	A	B	C	D		m_i	A	B	C	D	
2	0	0	1	0	✓	2,6	0	—	1	0	P_2	8,9	1	—	0	—	P_1
4	0	1	0	0	✓	2,10	—	0	1	0	P_3	12,13					
8	1	0	0	0	✓	4,6	0	1	—	0	P_4						
6	0	1	1	0	✓	4,12	—	1	0	0	P_5						
9	1	0	0	1	✓	8,9	1	0	0	—	✓						
10	1	0	1	0	✓	8,10	1	0	—	0	P_6						
12	1	1	0	0	✓	8,12	1	—	0	0	✓						
13	1	1	0	1	✓	9,13	1	—	0	1	✓						
15	1	1	1	1	✓	12,13	1	1	0	—	✓						
						13,15	1	1	—	1	P_7						

③ 求函数的最小覆盖。

去掉表 3.18 中的 P_1 和 P_7 行, 以及已被 P_1 和 P_7 覆盖的最小项 m_8 、 m_9 、 m_{12} 、 m_{13} 和 m_{15} , 得到简化的质蕴涵表, 如表 3.16 所示。

利用前面讲的行列消去法或用代数法求得所需质蕴涵为 P_3 和 P_4 , 因此, 函数的最简表达式为

$$F(A, B, C, D) = P_1 + P_7 + P_3 + P_4 = A\bar{C} + ABD + \bar{B}C\bar{D} + \bar{A}B\bar{D}$$

表 3.18 质蕴涵表

	2	4	6	8	9	10	12	13	15
✓ P_1				×	⊗		×	×	
P_2	×		×						
P_3	×					×			
P_4		×	×						
P_5		×					×		
P_6				×		×			
✓ P_7								×	⊗
覆盖情况				×	×		×	×	×

【例 3.18】化简不完全确定的布尔函数

$$F(A, B, C, D) = \sum m(1, 3, 4, 5, 10, 11, 12) \\ + \sum d(2, 13)$$

对于不完全确定的布尔函数的化简, 要注意两点: 一是在列表求所有质蕴涵时, 应该令 $d=1$, 以尽量利用随意项进行合并; 二是在列质蕴涵表时, 应令 $d=0$, 即随意项的覆盖问题可不必考虑, 以有利于得到最简式。

① 求所有质蕴涵项。令所有随意项为 1, 如表 3.19 所示。

表 3.19 求质蕴涵项 P_i

m_i	A	B	C	D		m_i	A	B	C	D		m_i	A	B	C	D	
1	0	0	0	1	✓	1,3	0	0	—	1	P_3	2,3	—	0	1	—	P_1
2	0	0	1	0	✓	1,5	0	—	0	1	P_4	10,11	—	0	1	—	
4	0	1	0	0	✓	2,3	0	0	1	—	✓	4,5	—	1	0	—	P_2
3	0	0	1	1	✓	2,10	—	0	1	0	✓	12,13					
5	0	1	0	1	✓	4,5	0	1	0	—	✓						
10	1	0	1	0	✓	4,12	—	1	0	0	✓						
12	1	1	0	0	✓	3,11	—	0	1	1	✓						
11	1	0	1	1	✓	5,13	—	1	0	1	✓						
13	1	1	0	1	✓	10,11	1	0	1	—	✓						
						12,13	1	1	0	—	✓						

求得所有质蕴涵项为

$$P_1 = \bar{B}C, \quad P_2 = B\bar{C}, \quad P_3 = \bar{A}BD, \quad P_4 = \bar{A}\bar{C}D$$

② 求必要质蕴涵。令所有的随意项为 0, 即在质蕴涵表中不必列随意项, 如表 3.20 所示。

由表 3.20 可确定必要质蕴涵为 P_1 和 P_2 。

③ 求函数的最小覆盖。

去掉质蕴涵表中被必要质蕴涵覆盖的最小项后, 只剩下最小项 m_1 未被覆盖, 直接从表 3.20 可以选取 P_3 或 P_4 作为所需质蕴涵。因此, 函数的最简表达式为

$$F(A, B, C, D) = P_1 + P_2 + P_3 = \bar{B}C + B\bar{C} + \bar{A}BD$$

或者 $F(A, B, C, D) = P_1 + P_2 + P_4 = \bar{B}C + B\bar{C} + \bar{A}\bar{C}D$

表 3.20 质蕴涵表

$m_i \backslash P_j$	1	3	4	5	10	11	12
$\checkmark P_1$		×			⊗	⊗	
$\checkmark P_2$			⊗	×			⊗
P_3	×	×					
P_4	×			×			
覆盖情况		×	×	×	×	×	×

以上介绍的都是单个布尔函数的化简, 然而在逻辑设计中, 经常会遇到多输出布尔函数。所谓多输出布尔函数, 就是同样的布尔变量输入得到多个输出的函数。我们当然可以利用卡诺图或列表法单独地化简每个输出函数。然而有时这样得到的电路不一定是最简单的。有时被某个输出函数使用的一些项可以被其他输出函数共享, 从而可以减少总的门数。因此我们在化简多输出函数时要考虑尽量多地找出所有的输出函数的公共项, 而不是仅仅考虑单个输出函数的化简。

习题 3

3.1 下列函数当变量(A, B, C, ...)取哪些 1 时, F 的值为 1:

$$(1) F = AB + \bar{A}C$$

$$(2) F = A\bar{B} + \bar{A}B$$

$$(3) F = \bar{A}\bar{B} + AB$$

$$(4) F = ABC + ABC\bar{C} + A\bar{B}C + \bar{A}BC$$

$$(5) F = (A + \bar{B} + \bar{A}B)(A + \bar{B})\bar{A}B$$

$$(6) F = (A + B + C)(A + B + \bar{C})(A + \bar{B} + C)(A + \bar{B} + \bar{C})$$

$$(7) F = (A + \bar{B}\bar{C})\bar{D} + (A + \bar{B})CD$$

$$(8) F = (A \oplus B)C + \bar{A}(B \oplus C)$$

3.2 用真值表验证下列等式:

$$(1) \overline{A+B} = \bar{A} \cdot \bar{B}$$

$$(2) A\bar{B} + \bar{A}B = (\bar{A} + \bar{B})(A + B)$$

$$(3) (\bar{A} + \bar{B})(A + B) = \overline{AB + \bar{A}\bar{B}}$$

$$(4) AB + A\bar{B} + \bar{A}B + \bar{A}\bar{B} = 1$$

$$(5) A \oplus B \oplus C = A \odot B \odot C$$

$$(6) (A \oplus B)C = A \oplus (B \oplus C)$$

3.3 用基本公式和基本规则证明下列等式:

$$(1) BC + D + \bar{D}(\bar{B} + \bar{C})(AD + B) = B + D$$

$$(2) AB + A\bar{B} + \bar{A}B + \bar{A}\bar{B} = 1$$

$$(3) (A + B)(A + \bar{B})(\bar{A} + B)(\bar{A} + \bar{B}) = 0$$

$$(4) ABC + \bar{A}\bar{B}\bar{C} = \overline{A\bar{B} + B\bar{C} + \bar{A}C}$$

$$(5) A\bar{B} + B\bar{C} + C\bar{A} = \overline{A\bar{B} + B\bar{C} + \bar{C}A}$$

$$(6) AB + BC + AC = (A + B)(B + C)(A + C)$$

$$(7) (AB + \bar{A}\bar{B})(BC + \bar{B}\bar{C})(CD + \bar{C}\bar{D}) = \overline{A\bar{B} + B\bar{C} + C\bar{D} + D\bar{A}}$$

$$(8) A \oplus B \oplus C = A \odot B \odot C$$

$$(9) (x \oplus y) \oplus z = x \oplus (y \oplus z)$$

3.4 用展开定理将下式各式简化成为 $A(\dots) + \bar{A}(\dots)$ 形式及 $[A + (\dots)][\bar{A} + (\dots)]$ 形式, 括号中 A 及 $\bar{A}$ 均不出现:

$$(1) F = AG + (A + B)C + \bar{A}D + (\bar{A} + F)E$$

$$(2) F = (A + \bar{B})(\bar{A} + C)(\bar{D} + E + AF)(G + \bar{H} + \bar{A}J)$$

3.5 写出下列表达式的对偶式:

$$(1) F = (A + B)(\bar{A} + C)(C + DE) + F$$

$$(2) F = \overline{\bar{A}\bar{B} \cdot \bar{C}\bar{D} \cdot \bar{D}\bar{A}\bar{B}}$$

$$(3) F = \overline{\bar{A} + B + \bar{B} + C + A + C + \bar{B} + C}$$

$$(4) F = B(\bar{A} \oplus \bar{B}) + B(A \oplus C)$$

$$(5) F = \overline{(C \odot A) \oplus (B \oplus \bar{D})}$$

3.6 求下列函数的补函数:

$$\checkmark (1) F = [(\bar{x}_1 x_2 + \bar{x}_3)x_4 + \bar{x}_5]x_6$$

$$(2) F = S[\bar{W} + I(T + \bar{C})] + H$$

$$(3) F = A[\bar{B} + (\bar{C}\bar{D} + \bar{E}F)G]$$

$$(4) F = A\bar{B} + B\bar{C} + C(\bar{A} + D)$$

3.7 将下列函数转换为由“标准积之和”形式表示的函数:

$$\checkmark (1) F(A, B, C) = A\bar{B}C + \bar{A}B + AC + AB\bar{C}$$

$$\checkmark (2) F(A, B, C) = \bar{A} + A\bar{C} + BC + A\bar{B}C$$

$$(3) F(A, B, C) = B(A + \bar{B} + \bar{C})(\bar{A} + \bar{C})C$$

$$(4) F(A, B, C) = ABC + \bar{A}\bar{B}\bar{C}$$

$$(5) F(A, B, C) = (\bar{A}B + C)[(\bar{A}\bar{B} + B)C + A]$$

3.8 用“标准和之积”形式表示 3.7 题中的函数。

3.9 用代数运算法求下列各函数的“标准积之和”形式及“标准和之积”形式：

$$(1) F(A, B, C, D) = ABC\bar{D} + ABCD$$

$$(2) F(A, B, C, D) = \bar{A}BCD + \bar{A}\bar{B}CD + ACD + \bar{B}CD$$

$$(3) F(A, B, C, D) = B\bar{C}\bar{D} + \bar{A}B + AB\bar{C}D + BC$$

$$(4) F(A, B, C) = \bar{C} + \bar{A}B + \bar{B}C + ABC$$

$$(5) F(A, B, C, D) = \bar{A}(\bar{B} + C)(A + \bar{C})(A + B + C + D)(A + C + \bar{D})$$

3.10 设 $F(A, B, C, D, E) = \sum m(0, 1, 3, 7, 8, 9, 12, 15, 16, 17, 20, 28, 29, 30, 31)$ ，试用最大项表示这个函数。

3.11 求 3.10 题中所给函数的补函数，并以最大项表示之。

3.12 证明 $\bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + \bar{A}BC + ABC = A \oplus B \oplus C$ 。

3.13 设 A, B, C 为逻辑变量，试回答：

(1) 若已知 $A + B = A + C$ ，则 $B = C$ ，对吗？

(2) 若已知 $AB = AC$ ，则 $B = C$ ，对吗？

(3) 若已知 $A + B = A + C$ ，且 $AB = AC$ ，则 $B = C$ ，对吗？

3.14 用代数化简法将下列函数化简为“与或”表达式：

$$(1) F = \bar{A}\bar{B}C + \bar{A}BC + ABC + ABC \quad \text{最简}$$

$$(2) F = A\bar{B} + B + BCD$$

$$(3) F = ABC + \bar{A} + \bar{B} + \bar{C}$$

$$(4) F = \bar{A}\bar{B} + (AB + \bar{A}\bar{B} + \bar{A}B)C$$

$$(5) F = \overline{A[B + \bar{C}(D + E)]}$$

$$(6) F = \overline{(A + BC)(\bar{A} + \bar{D}\bar{E})} = A'B' + A'C + AD + AE$$

$$(7) F = (X + Y + Z + \bar{W})(V + X)(\bar{V} + Y + Z + \bar{W})$$

$$(8) F = \overline{ABC + \bar{A}\bar{B} + BC}$$

$$(9) F = \overline{ABC(A + B + C)}$$

$$(10) F = \overline{(\bar{A}\bar{B} + ABC)(\bar{A}\bar{B}C)}$$

3.15 用卡诺图法将下列函数化为最简“与或”表达式：

$$(1) F(A, B, C) = \sum m(0, 1, 2, 4, 5, 7)$$

$$(2) F(A, B, C, D) = \sum m(0, 1, 2, 3, 4, 6, 7, 8, 9, 11, 15)$$

$$(3) F(A, B, C, D) = \sum m(3, 4, 5, 7, 9, 13, 14, 15)$$

$$(4) F(A, B, C, D) = \sum m(0, 1, 2, 5, 6, 7, 8, 9, 13, 14)$$

$$(5) F(A, B, C, D) = \sum m(0, 2, 3, 5, 7, 8, 10, 11) + \sum d(14, 15)$$

$$(6) F(A, B, C, D, E) = \sum m(0, 3, 4, 6, 7, 8, 11, 15, 16, 17, 20, 22, 25, 27, 29, 30, 31)$$

$$(7) F(A, B, C, D, E) = \sum m(0, 1, 2, 3, 4, 5, 6, 7, 16, 17, 20, 21) + \sum d(24, 25, 27, 28, 30)$$

$$(8) F(A, B, C, D, E) = \prod M(0, 1, 2, 3, 8, 9, 16, 17, 20, 21, 24, 25, 28, 29, 30, 31) \cdot \prod d(13, 14, 19)$$

3.16 用 Q-M 法求下列函数的所有质蕴涵:

$$(1) F(A, B, C, D) = \sum m(0, 1, 2, 3, 6, 7, 8, 9, 10, 13, 14, 15)$$

$$(2) F(A, B, C, D) = \sum m(0, 1, 2, 3, 6, 7, 9, 10, 14, 15) + \sum d(8, 13)$$

$$(3) F(A, B, C, D, E) = \sum m(0, 2, 4, 9, 11, 14, 15, 16, 17, 19, 23, 25, 29, 31)$$

3.17 求下列函数的最简“或与”式:

$$(1) F(A, B, C, D) = \sum m(4, 5, 6, 13, 14, 15)$$

$$(2) F(A, B, C, D) = \sum m(4, 5, 6, 13, 14, 15) + \sum d(8, 9, 10)$$

3.18 某单位有 5 位外语人员: A 会英语和法语, B 会英语和俄语, C 会俄语和日语, D 会德语, E 会日语和法语。

(1) 外地有一外事活动, 要求英、俄、日、德、法 5 种外语, 求最经济的出差方案。

(2) 试写出这 5 人中两两进行外语会话的条件(这里指只有两人在场的会话)。

3.19 已知 $F = F(A, B, C, D)$ 的全部质蕴涵为 $\overline{A}\overline{B}\overline{C}$ 、 $\overline{A}\overline{C}\overline{D}$ 、 $\overline{B}D$ 、 CD 、 $\overline{A}B\overline{D}$ 、 BC 、 AD 、 AC 。求 F 的最简与或式。要求: 列质蕴涵表, 找必要质蕴涵, 列简化的质蕴涵表, 找最小质蕴涵覆盖。

3.20 用 Q-M 法化简下列函数:

$$(1) F(A, B, C, D, E) = \prod M(0, 1, 2, 3, 8, 9, 10, 11, 17, 19, 21, 23, 25, 27, 29, 31)$$

$$(2) F(A, B, C, D, E) = \prod M(0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 25, 27, 29, 31) \cdot \prod d(5, 7, 13, 15, 24, 26, 28, 30)$$

3.21 用公式和定理化简下列函数:

$$(1) F(A, B, C) = ABC + \overline{A}BC + \overline{A}B\overline{C} + A\overline{B}\overline{C} + \overline{A}\overline{B}\overline{C}$$

$$(2) F(A, B, C) = AC + ABC + A\overline{C} + \overline{A}\overline{B}\overline{C} + BC$$

3.22 用卡诺图化简如下函数, 并列出它们的质蕴涵项和必要质蕴涵项:

$$(1) F(x_1, x_2, x_3, x_4) = \sum m(0, 1, 4, 7, 9, 10, 13) + \sum d(2, 5, 8, 12, 15)$$

$$(2) F(X_1, X_2, X_3, X_4) = \prod M(0, 13, 15) \cdot \prod d(3, 7, 9, 10, 12, 14)$$

3.23 用卡诺图化简如下四变量函数:

$$F(A, B, C, D) = F_1(A, B, C, D) \oplus F_2(A, B, C, D)$$

其中, $F_1 = \overline{A}D + BC + \overline{B}\overline{C}\overline{D} + \sum d(2, 11, 13)$

$$F_2 = \prod M(0, 2, 4, 8, 9, 10, 14) \cdot \prod d(1, 7, 13, 15)$$

3.24 用两函数卡诺图之对应单元相加的方法求两函数的布尔和 $F_1 + F_2$ 。其中

$$F_1(x_1, x_2, x_3, x_4) = \sum m(0, 3, 10, 12, 15) + \sum d(5, 6, 9)$$

$$F_2(x_1, x_2, x_3, x_4) = \sum m(0, 2, 5, 10, 15) + \sum d(1, 6, 12)$$

3.25 用两函数卡诺图的对应单元相乘的方法求两函数的布尔积 $F_1 \cdot F_2$ 。其中

$$F_1(x_1, x_2, x_3, x_4) = \sum m(0, 2, 12, 14) + \sum d(3, 5, 9, 15)$$

$$F_2(x_1, x_2, x_3, x_4) = \sum m(5, 6, 10, 12) + \sum d(1, 2, 8, 9, 15)$$

3.26 化简下列多输出函数:

✓(1) $F_1(A, B, C, D) = A + D + \bar{A}\bar{C}D$

$$F_2(A, B, C, D) = \bar{C}\bar{D} + ABD + \bar{B}\bar{C}D$$

$$F_3(A, B, C, D) = \bar{B}\bar{D} + ABCD + \bar{A}\bar{B}\bar{C}$$

(2) $F_1(A, B, C, D) = \sum m(2, 3, 4, 5, 6, 7, 11, 14) + \sum d(9, 10, 13, 15)$

$$F_2(A, B, C, D) = \sum m(0, 1, 3, 4, 5, 7, 11, 14) + \sum d(8, 10, 12, 13)$$

第4章 组合逻辑电路

通常将数字系统的逻辑电路分为两大类,一类称为组合逻辑电路,另一类称为时序逻辑电路。所谓组合逻辑电路,是指没有从输出到输入的反馈,且由功能完全的门系列构成的电路,即不含记忆元件(能存储信息,如触发器)的逻辑电路是组合逻辑电路。包含记忆元件的逻辑电路是时序逻辑电路。本章主要讨论组合逻辑的相关知识。组合逻辑的结构模型如图4.1所示。设 X 是所有输入变量($x_1, x_2, \dots, x_n$)的集合, Y 是所有输出变量($y_1, y_2, \dots, y_n$)的集合, $Y=F(X)$ 。输出不向输入反馈。组合逻辑电路也称为组合网络。

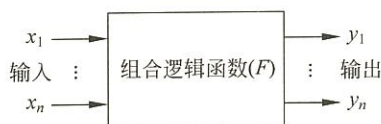


图 4.1 组合逻辑模型

本章在前面章节的基础上讲述如何分析和设计组合逻辑电路。主要讨论中、小规模的组合电路的分析和设计方法,以及介绍一些常用的组合电路。

4.1 常用逻辑门的图形符号

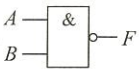
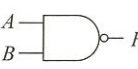
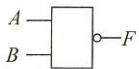
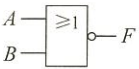
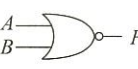
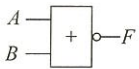
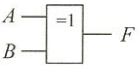
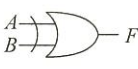
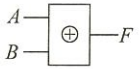
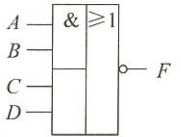
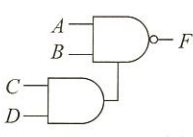
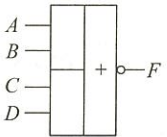
在数字逻辑电路中,把能实现基本逻辑运算的单元电路称为逻辑门电路,包括基本的与门、或门、非门以及复合逻辑与非、或非和与或非门等,这些都是构成数字系统最核心的单元。

在讨论组合电路的分析和设计方法之前,我们给出常用逻辑门的图形符号汇总图,如表4.1所示,包括我国最新的国标符号、国际常用的图形符号以及我国较早的部颁符号,方便大家查阅。本书统一采用国标符号。

表 4.1 常用逻辑门的图形符号

逻辑门	国标符号	国际常用符号	我国部颁符号	输出表达式
“与”门				$F=A \cdot B$
“或”门				$F=A+B$
“非”门				$F=\bar{A}$

续表

逻辑门	国标符号	国际常用符号	我国部颁符号	输出表达式
“与非”门				$F = \overline{A \cdot B}$
“或非”门				$F = \overline{A + B}$
“异或”门				$F = A \oplus B = A\bar{B} + \bar{A}B$
“与或非”门				$F = \overline{AB + CD}$

4.2 布尔函数的实现

大家已经知道,布尔函数和逻辑电路之间存在一一对应关系。布尔函数表达式的基本运算是“与”、“或”、“非”,因而,任何布尔函数都可以用与门、或门、非门所构成的逻辑电路来实现。但是,随着半导体集成电路的产生和发展,实际的逻辑电路已不仅用与门、或门、非门作为基本逻辑单元,也采用复合门作为基本逻辑门。常用的复合门电路有与非门、或非门和与或非门等。

下面分别介绍用与非门、或非门和与或非门实现布尔函数的方法。

4.2.1 用与非门实现布尔函数

用与非门实现布尔函数时,首先将函数化成最简“与或”形式;然后再对表达式二次取反,得到函数的“与非-与非”表达式;最后用与非门实现。

【例 4.1】 用与非门实现函数

$$F(A, B, C, D) = AB\bar{D} + AC + A\bar{C}D + AD$$

首先,将函数化成最简“与或”形式。画出函数的卡诺图,如图 4.2(a)所示。由卡诺图化简得表达式

$$F(A, B, C, D) = AB + AC + AD$$

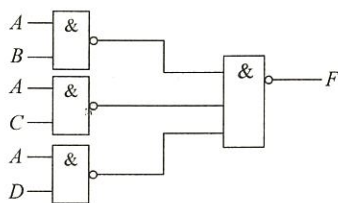
再将上述表达式二次取反,则有

$$F(A, B, C, D) = \overline{\overline{AB + AC + AD}} = \overline{\overline{AB} \cdot \overline{AC} \cdot \overline{AD}}$$

这是函数的二级“与非”表达式。用与非门来实现该表达式,其逻辑电路图如图 4.2(b)所示。

AB \ CD	00	01	11	10
00	0	0	1	0
01	0	0	1	1
11	0	0	1	1
10	0	0	1	1

(a) 卡诺图



(b) 用与非门实现的逻辑电路图

图 4.2 用与非门实现布尔函数示例

4.2.2 用或非门实现布尔函数

用或非门实现布尔函数时,首先将函数化成最简“或与”形式;然后再对表达式二次取反,得到函数的“或非-或非”表达式;最后用或非门画出逻辑图。

【例 4.2】 用或非门实现函数

$$F(A, B, C, D) = AB\bar{D} + AC + A\bar{C}D + AD$$

首先将函数化简成“或与”形式。画出函数的卡诺图,如图 4.3(a)所示。图中,标 1 方格表示原函数 F 的各个最小项,而标 0 方格表示补函数 $\bar{F}$ 的各个最小项(或原函数的最大项)。因此,求得补函数 $\bar{F}$ 的最简“与或”式为

$$\bar{F}(A, B, C, D) = \bar{A} + \bar{B}\bar{C}\bar{D}$$

再对该式两边取一次反,即可求得函数 F 的“或与”表达式为

$$F(A, B, C, D) = A(B + C + D)$$

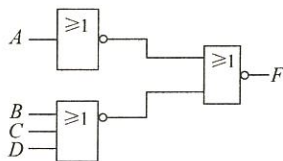
然后,再对上述表达式二次取反,即可得函数的“或非-或非”表达式为

$$F(A, B, C, D) = \overline{\overline{A(B + C + D)}} = \overline{\bar{A} + \bar{B} + \bar{C} + \bar{D}}$$

最后,用或非门来实现该表达式,其逻辑电路如图 4.3(b)所示。

AB \ CD	00	01	11	10
00	0	0	1	0
01	0	0	1	1
11	0	0	1	1
10	0	0	1	1

(a) 卡诺图



(b) 用或非门实现的逻辑图

图 4.3 用或非门实现布尔函数示例

4.2.3 用与或非门实现布尔函数

用与或非门实现逻辑函数时,首先将函数化简成原函数 F 的最简“与或”式以及补函数

$\bar{F}$ 的最简“与或”式；然后再对 F 的表达式二次取反,对 $\bar{F}$ 的表达式一次取反,即得 F 的两个“与或非”表达式；最后用与或非门实现该函数,比较两者,取较简单的一个。

【例 4.3】 用与或非门实现函数

$$F(A, B, C, D) = ABD + AC + A\bar{C}D + AD$$

首先求 F 和 $\bar{F}$ 的最简“与或”式。由前面图 4.2(a),求得 F 的最简“与或”式为

$$F(A, B, C, D) = AB + AC + AD$$

再由前面的图 4.3(a),求得 $\bar{F}$ 的最简“与或”式为

$$\bar{F}(A, B, C, D) = \bar{A} + \bar{B}\bar{C}\bar{D}$$

然后,对 F 的最简式二次取反,则得

$$F(A, B, C, D) = \overline{\bar{A}B + AC + AD}$$

再对补函数 $\bar{F}$ 的最简式一次取反,则得

$$F(A, B, C, D) = \overline{\bar{A} + \bar{B}\bar{C}\bar{D}}$$

最后,用与或非门来实现上述两个表达式,其逻辑图如图 4.4 所示。比较两者,可见图 4.4(b)比图 4.4(a)简单。但是,如果有时要求同时得到 F 及 $\bar{F}$ 时,也可采用 4.4(a)的方案。

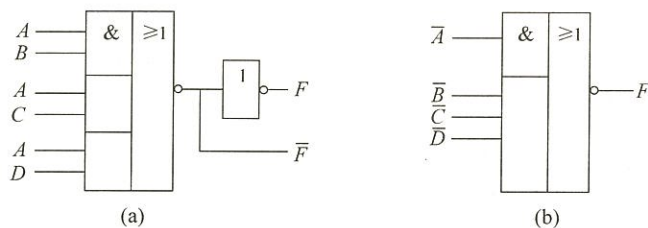


图 4.4 用与或非门实现布尔函数示例

4.3 组合电路的分析

当人们研究某一给定的逻辑电路时,常常会遇到这样一些问题:需要采取电路的设计思想;或者需要更换电路的某些组件;或者需要评价电路的技术经济指标等。这就要求我们对给定的逻辑电路进行分析。

组合网络的分析方法,一般可概括为以下几个步骤。

- (1) 根据给定的逻辑电路图,写出布尔函数表达式。
- (2) 将得到的布尔函数表达式化简。
- (3) 由简化的布尔函数表达式列出真值表。
- (4) 判断该电路所能完成的逻辑功能,作出简要的文字描述,或进行改进设计。

下面举例说明组合逻辑网络的分析过程。

【例 4.4】 分析图 4.5 所示逻辑电路的逻辑功能。

① 由图写出布尔函数表达式。

为了分析方便,可先写出各个门的输出表达式,再

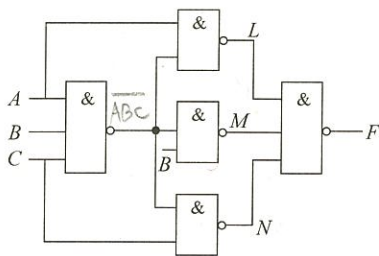


图 4.5 非一致电路的逻辑图

写出总的布尔表达式,则有

$$L = A \cdot \overline{ABC}, \quad M = B \cdot \overline{ABC}, \quad N = C \cdot \overline{ABC}$$

$$F = \overline{L \cdot M \cdot N} = \overline{L} + \overline{M} + \overline{N} = A \cdot \overline{ABC} + B \cdot \overline{ABC} + C \cdot \overline{ABC}$$

② 化简表达式。

$$\begin{aligned} F &= A \cdot \overline{ABC} + B \cdot \overline{ABC} + C \cdot \overline{ABC} \\ &= A \cdot (\overline{A} + \overline{B} + \overline{C}) + B \cdot (\overline{A} + \overline{B} + \overline{C}) + C(\overline{A} + \overline{B} + \overline{C}) \\ &= A\overline{B} + A\overline{C} + \overline{A}B + B\overline{C} + \overline{A}C + \overline{B}C \end{aligned}$$

用卡诺图化简法,如图 4.6 所示。最后,可得

$$F = A\overline{B} + B\overline{C} + \overline{C}A$$

或

$$F = \overline{A}B + \overline{B}C + \overline{C}A$$

③ 列真值表,如表 4.2 所示。

		AB			
		00	01	11	10
C	0	0	1	1	1
	1	1	1	0	1

图 4.6 非一致电路的卡诺图

表 4.2 非一致电路的真值表

ABC	F
000	0
001	1
010	1
011	1
100	1
101	1
110	1
111	0

④ 由真值表可知,只要输入 A、B、C 的取值不一样,输出 F 就为 1; 否则,当 A、B、C 取值一样时, F 为 0,所以这是一个三变量的非一致电路。电路无反变量输入,这是它的特点。

【例 4.5】 分析图 4.7(a)所示的逻辑电路。

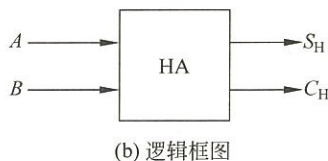
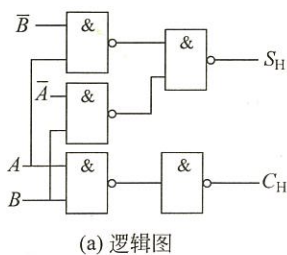


图 4.7 半加器

① 由图 4.7(a)写出布尔函数表达式。

$$S_H = \overline{A}B + A\overline{B} = A \oplus B, \quad C_H = \overline{A}B = AB$$

② 化简函数。函数形式已是最简。

③ 列真值表,如表 4.3 所示。

④ 电路逻辑功能的描述。由真值表可知,该电路是求 A 、 B 的和以及进位,分别是 S_H 、 C_H 。

我们把能对两个一位二进制数进行相加而求得“和”及“进位”的逻辑电路称为半加器。它的逻辑框图如图 4.7(b)所示。其中, A 、 B 分别为两个一位二进制数的输入; S_H 、 C_H 分别为相加形成的“和”及“进位”。半加器还可以用异或门、与门实现,逻辑图如图 4.8 所示。

表 4.3 半加器的真值表

A	B	S_H	C_H
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

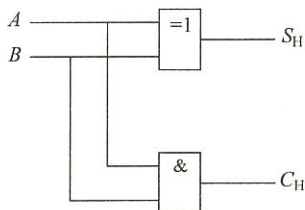


图 4.8 用异或门、与门实现半加器的逻辑图

4.4 组合电路的设计

组合逻辑电路的设计过程正好与分析过程相反,它是根据给定的逻辑功能要求,找出用最少逻辑门来实现该逻辑功能的电路。

组合逻辑电路的设计,一般可分为以下几个步骤。

- (1) 根据设计的逻辑要求列出真值表。
- (2) 根据真值表写出布尔函数表达式。
- (3) 化简函数表达式。
- (4) 根据给定的逻辑门,画出逻辑图。□

这 4 个设计步骤中,最关键的是第一步,即根据逻辑要求列真值表。任何逻辑问题,只要能列出它的真值表,就能把逻辑电路设计出来。然而,由于逻辑要求往往是用文字描述的,一般很难做到全面而确切,有时甚至是含糊不清的。因此,对设计者来说建立真值表不是一件很容易的事,它要求设计者对设计的逻辑问题有一个全面的理解,对每一种可能的情况都能作出正确的判断。列真值表的困难犹如算术中把文字题列成算式一样。

要列真值表,首先必须从对问题的文字描述中搞清什么作为变量,什么作为函数,以及它们之间的关系。然后,采用穷举法,列出变量可能出现的所有情况,逐一加以分析,最后综合起来即为真值表。

下面举几个例子,说明组合逻辑电路的设计过程。

【例 4.6】 设两个一位二进制数为 x_1 和 y_1 , 试设计比较器,如 $x_1 > y_1$, 输出 1, 否则输出 0。

① 根据逻辑要求列真值表。

显然,这里变量为 x_1 和 y_1 , 而 $x_1 > y_1$ 比较结果为输出,根据题意,列出真值表如表 4.4 所示。

② 由真值表写出函数表达式: $F = \sum m(2) = x_1 \overline{y_1}$, 该式已为最简。

③ 画出逻辑电路图。如用与门来实现,且输入信号中原变量和反变量都存在,电路图如图 4.9 所示。

表 4.4 $x_1 > y_1$ 的真值表

x_1	y_1	$F(x_1, y_1)$
0	0	0
0	1	0
1	0	1
1	1	0



图 4.9 $x_1 > y_1$ 比较器的逻辑图

【例 4.7】 设 x 和 y 是两个两位的二进制数,其中 $x = x_1x_2, y = y_1y_2$ 。试设计比较器,如 $x > y$,输出 1,否则输出 0。

① 根据逻辑要求列真值表。这个问题的变量有 4 个,即 x_1, x_2, y_1, y_2 ,而 $x > y$ 比较结果为输出。比较两个数的大小,先从高位开始,即先比较 x_1, y_1 ,如果 $x_1 = 1, y_1 = 0$,则不管 x_2, y_2 怎样,肯定 $x > y$,即 $F = 1$;反之, $x_1 = 0, y_1 = 1$,肯定 $x < y$,即 $F = 0$;只有当 $x_1 = y_1$ 时,才需要再比较 x_2, y_2 ,当 $x_2 > y_2$ 时, $x > y$ 成立,即 $F = 1$ 。

通过以上分析,可列出真值表,如表 4.5 所示。这是一个部分真值表,表中只列出使 F 为 1 的那些输入组合。 d 表示变量可取 0,也可取 1。

表 4.5 $x_1x_2 > y_1y_2$ 的真值表

x_1	y_1	x_2	y_2	$F(x_1, y_1)$
1	0	d	d	1
0	0	1	0	1
1	1	1	0	1

② 由真值表写出函数表达式: $F = \sum m(2, 8, 9, 10, 11, 14)$,卡诺图如图 4.10 所示。

③ 化简:由卡诺图化简得 $F = x_1\overline{y_1} + x_2\overline{y_1}\overline{y_2} + x_1x_2\overline{y_2}$ 。

④ 画出逻辑电路图。如用与非门来实现,且输入信号中原变量和反变量都存在, $F = \overline{\overline{x_1y_1} \cdot \overline{x_2y_1y_2} \cdot \overline{x_1x_2y_2}}$,电路图如图 4.11 所示。

x_1y_1	00	01	11	10
x_2y_2				
00				1
01				1
11				1
10	1		1	1

图 4.10 例 4.7 的卡诺图

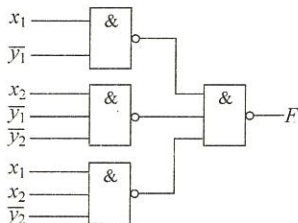


图 4.11 用与非门实现 $x > y$ 比较器的逻辑图

【例 4.8】 设计一个操作码形成器,如图 4.12 所示。当按下 +、-、 $\times$ 各个操作键时,要求分别产生加法、减法和乘法的操作码 01、10 和 11。

① 根据逻辑要求列真值表。

根据题意,所要设计的线路有 3 个输入,即 +、-、 $\times$ 3 个按键,分别用变量 A, B, C 来表

示；输出函数为 F_1 和 F_2 。当按下某一按键时，相应输入变量的取值为 1；否则，取值为 0。在正常操作下，每次只允许按下一个按键，而不允许同时按下两个或两个以上按键。因此，A、B、C 3 个变量中同时有两个或两个以上取值为 1 的情况，就作为随意项处理。

由以上分析，可列出表 4.6 所示的真值表。

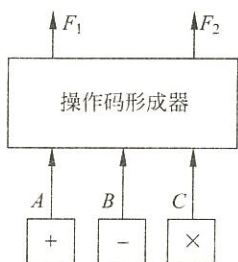


图 4.12 操作码形成器框图

表 4.6 操作码形成器真值表

A	B	C	F_1	F_2
0	0	0	0	0
0	0	1	1	1
0	1	0	1	0
0	1	1	d	d
1	0	0	0	1
1	0	1	d	d
1	1	0	d	d
1	1	1	d	d

② 由真值表写出函数表达式： $F_1 = \sum m(1,2) + \sum d(3,5,6,7)$ ， $F_2 = \sum m(1,4) + \sum d(3,5,6,7)$ ，卡诺图如图 4.13 所示。

③ 化简：由卡诺图化简得 $F_1 = B + C$ ， $F_2 = A + C$ 。

④ 画出逻辑电路图。如用或门来实现，电路图如图 4.14 所示。

$\begin{matrix} AB \\ C \end{matrix}$	00	01	11	10
0		1	d	
1	1	d	d	d

F_1

$\begin{matrix} AB \\ C \end{matrix}$	00	01	11	10
0			d	1
1	1	d	d	d

F_2

图 4.13 操作码形成器卡诺图

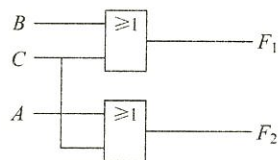


图 4.14 用或门实现操作码形成器的逻辑图

4.5 常用组合电路

4.5.1 加法器

数字计算机能进行各种信息处理，其中最常用的是各种算术运算，而加法是最基本的算术运算。实现二进制加法运算的逻辑电路，通常称为加法器，下面围绕加法器的设计进行讨论，并介绍一些常用的加法器件。

1. 全加器(Full Adder)

能对两个一位二进制数相加并考虑低位来的进位，即相当于 3 个一位二进制数的相加，得到“和”及“进位”的逻辑电路，称为全加器，它的框图如图 4.15 所示，其中， A_i 和 B_i 分别

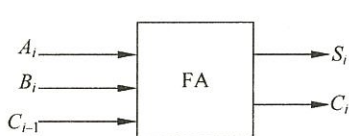


图 4.15 全加器逻辑框图

为两个一位二进制数的输入； C_{i-1} 为低位来的进位输入； S_i 和 C_i 分别为相加后形成的“和”及向高位的“进位”输出。

要设计一位全加器，设计步骤如下。

(1) 列真值表，如表 4.7 所示。

表 4.7 全加器真值表

A_i	B_i	C_{i-1}	S_i	C_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

(2) 写出函数的表达式：

$$S_i = \sum m(1, 2, 4, 7), \quad C_i = \sum m(3, 5, 6, 7)$$

(3) 化简。

画出 S_i 和 C_i 的卡诺图，如图 4.16 所示。

由卡诺图可得

$$S_i = \bar{A}_i \bar{B}_i C_{i-1} + \bar{A}_i B_i \bar{C}_{i-1} + A_i \bar{B}_i \bar{C}_{i-1} + A_i B_i C_{i-1}$$

$$C_i = A_i B_i + A_i C_{i-1} + B_i C_{i-1}$$

(4.1)

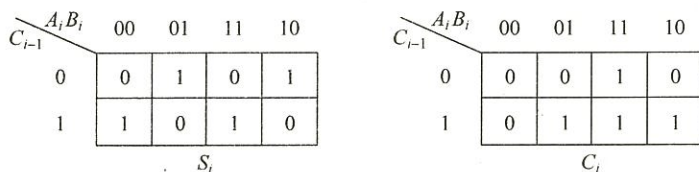


图 4.16 全加器的卡诺图

(4) 画逻辑图。

根据给定的不同类型门电路，将布尔函数 S_i 和 C_i 变换成不同形式。

方案一：用或非门实现全加器。将 S_i 和 C_i 用最简或与式表示，由图 4.16 卡诺图可得

$$\bar{S}_i = \bar{A}_i \bar{B}_i \bar{C}_{i-1} + \bar{A}_i B_i C_{i-1} + A_i \bar{B}_i \bar{C}_{i-1} + A_i \bar{B}_i C_{i-1}$$

$$\bar{C}_i = \bar{A}_i \bar{B}_i + \bar{A}_i \bar{C}_{i-1} + \bar{B}_i \bar{C}_{i-1}$$

(4.2)

式(4.2)取反后得：

$$S_i = (A_i + B_i + C_{i-1})(A_i + \bar{B}_i + C_{i-1})(\bar{A}_i + \bar{B}_i + C_{i-1})(\bar{A}_i + B_i + \bar{C}_{i-1})$$

$$C_i = (A_i + B_i)(A_i + C_{i-1})(B_i + C_{i-1})$$

(4.3)

比较式(4.1)和式(4.3)可发现，它们互为对偶式，即 $(S_i)_d = S_i, (C_i)_d = C_i$ ，这种函数称为自对偶函数。这是布尔函数中的特例。

由式(4.3)画逻辑图,如图4.17所示。

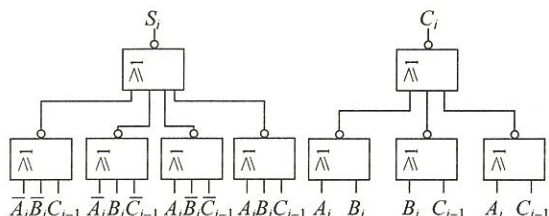


图 4.17 用或非门实现全加器的逻辑图

方案二: 用与或非门实现全加器。由式(4.2)可得:

$$\begin{aligned} S_i &= \overline{A_i B_i C_{i-1} + A_i B_i \bar{C}_{i-1} + A_i \bar{B}_i C_{i-1} + A_i \bar{B}_i \bar{C}_{i-1}} \\ C_i &= \overline{A_i B_i + A_i C_{i-1} + B_i C_{i-1}} \end{aligned} \quad (4.4)$$

根据式(4.4)可画出用与或非门构成的全加器,但这种全加器在构成多位加法器时,需加级间耦合门,如由低位的 C_{i-1} 经倒相器形成 $\bar{C}_{i-1}$,再形成 C_i ,这样增加了进位形成的时间。因此,这种全加器电路在实际中很少用。

下面介绍一种用与或非门的常用电路,其 S_i 和 C_i 的表达式如下。

$$\begin{aligned} \bar{S}_i &= \overline{A_i \bar{C}_i + B_i \bar{C}_i + C_{i-1} \bar{C}_i + A_i B_i C_{i-1}} \\ \bar{C}_i &= \overline{A_i B_i + A_i C_{i-1} + B_i C_{i-1}} \end{aligned} \quad (4.5)$$

$$\begin{aligned} S_i &= \overline{A_i C_i + \bar{B}_i C_i + \bar{C}_{i-1} C_i + \bar{A}_i \bar{B}_i \bar{C}_{i-1}} \\ C_i &= \overline{A_i \bar{B}_i + \bar{A}_i \bar{C}_{i-1} + \bar{B}_i \bar{C}_{i-1}} \end{aligned} \quad (4.6)$$

根据式(4.5)和式(4.6),用与或非门构成的全加器如图4.18所示。由图可见,当输入为原变量 A_i, B_i, C_{i-1} 时,输出为补函数 $\bar{S}_i, \bar{C}_i$; 当输入为反变量 $\bar{A}_i, \bar{B}_i, \bar{C}_{i-1}$ 时,输出为原函数 S_i, C_i 。用这种电路构成多位加法器时,可以省去级间耦合门,因此,这种方案电路简单,进位速度快,是一种非常简单而实用的方案。实现全加器的方案还有很多,例如用半加器、与非门等,这里不一一列举。

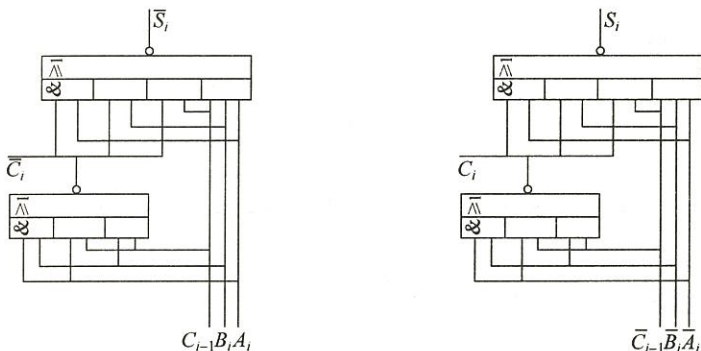


图 4.18 用与或非门实现全加器的逻辑图

2. 多位加法器

以上介绍了完成两个一位二进制数相加的逻辑电路。如果有两个 n 位二进制数相加,

就需 n 位全加器, 这样构成的逻辑电路称为多位并行加法器。按照进位方式的不同, 并行加法器分为行波进位加法器和先行进位加法器两种。下面分别讨论这两种加法器的设计。

(1) 行波进位加法器。行波进位加法器的逻辑框图如图 4.19 所示。这种加法器的构成比较简单, 只要把 n 位全加器串联起来, 低位全加器的进位输出连到相邻的高位全加器的进位输入。

由图 4.19 可见, 这样构成的加法器, 尽管各位相加是并行的, 但其进位信号是由低位向高位逐级传递的, 好像行波一样。这样, 要形成高位的和, 要等低位的进位形成后才能确定。正如我们做手算加法那样, 要从低位逐级求出进位, 最后才求得高位的和及进位。因此, 这种加法器, 由于进位是串行的, 形成进位的速度很慢, 加法器的速度主要受进位传递时间的限制。

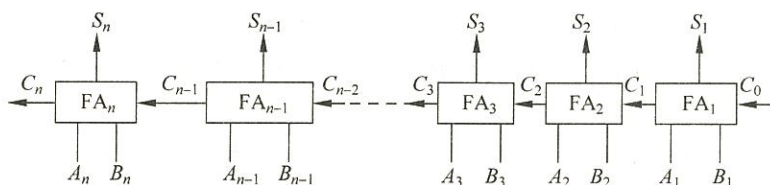


图 4.19 行波进位加法器

若每级全加器形成进位的延时为 $2t_{pd}$, 则在最坏情况下, 从 FA_1 的输入到产生高位进位 C_n 需要时间为 $2t_{pd} \times n$ 。当 n 增大时, 完成一次加法所需时间也随之增加。

(2) 先行进位加法器。为了提高加法速度, 在逻辑设计上采用所谓先行进位的方法, 即每一位的进位根据各位的输入同时预先形成, 而不需要等到低位的进位送来后才形成。

先行进位的原理是这样的: 根据进位表达式

$$C_i = A_i B_i + (A_i \oplus B_i) C_{i-1}$$

可以知道, 进位由两部分组成, $A_i B_i$ 表示当 $A_i = 1$ 且 $B_i = 1$ 时, 进位 $C_i = 1$, 它只与本位的输入 A_i 、 B_i 有关, 与低位来的进位 C_{i-1} 无关; $(A_i \oplus B_i) C_{i-1}$ 表示当 A_i 、 B_i 中有一个为 1 时, 当低位来进位信号, 则 $C_i = 1$ 。我们令

$$G_i = A_i B_i, \quad P_i = (A_i \oplus B_i) C_{i-1}$$

则

$$C_i = G_i + P_i C_{i-1} \quad (4.7)$$

我们称 G_i 为第 i 位的进位生成项; 称 P_i 为进位传递条件。同时, 可以把 S_i 写成

$$S_i = A_i \oplus B_i \oplus C_{i-1} = P_i \oplus C_{i-1} \quad (4.8)$$

式(4.7)和式(4.8)是先行进位的并行加法器的两个基本公式。由这两个公式, 可以写出各位全加器的表达式, 即有

$$\begin{cases} S_1 = P_1 \oplus C_0 \\ C_1 = G_1 + P_1 C_0 \\ S_2 = P_2 \oplus C_1 \\ C_2 = G_2 + P_2 C_1 = G_2 + P_2 G_1 + P_2 P_1 C_0 \end{cases}$$

$$\begin{cases}
 S_3 = P_3 \oplus C_2 \\
 C_3 = G_3 + P_3 C_2 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 C_0 \\
 S_4 = P_4 \oplus C_3 \\
 C_4 = G_4 + P_4 C_3 = G_4 + P_4 G_3 + P_4 P_3 G_2 + P_4 P_3 P_2 G_1 + P_4 P_3 P_2 P_1 C_0 \\
 \dots\dots \\
 S_n = P_n \oplus C_{n-1} \\
 C_n = G_n + P_n C_{n-1} = G_n + P_n G_{n-1} + P_n P_{n-1} G_{n-2} + \dots \\
 \quad + P_n P_{n-1} \dots P_2 G_1 + P_n P_{n-1} \dots P_2 P_1 C_0
 \end{cases}$$

其中 C_0 为来自外部进位输入; $G_i = A_i B_i$, $P_i = A_i \oplus B_i$ ($i=1, 2, \dots, n$) 是各位的进位生成项和进位传递条件, 由各位的数据输入确定。

由这些表达式, 可画出 n 位先行进位加法器的逻辑图。图 4.20 为四位先行进位加法器的逻辑图。从图可见, 利用先行进位方法, 各位进位都只经过三级门延时(约 $4t_{pd}$, 假定异或门延时为 $2t_{pd}$), 而形成各位和 S_i 需经四级门延时(约 $6t_{pd}$), 即先形成进位, 再形成和。

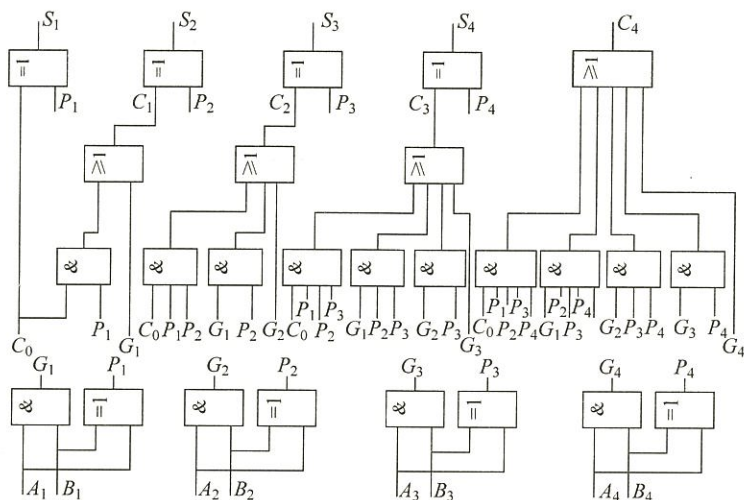


图 4.20 4 位先行进位加法器

显然, 进位传递时间的节省是以逻辑电路的复杂性为代价的。随着位数的增加, 所需电路元件也迅速增加, 而且门电路的扇入和扇出数也会增大。扇入和扇出是反映门电路的输入端数目和输出驱动能力的指标。扇入是指一个门电路所能允许的输入端个数。扇出是指一个门电路所能驱动的同类门的数目。因此, 当位数较多时, 为避免门电路的扇入扇出增大的问题, 通常采用折中的办法, 即将字长 n 位分为若干组。例如, 一个 16 位并行加法器, 电路可分为四组, 每组 4 位。每组内采用先行进位, 组间采用串行进位。

4.5.2 十进制数字的七段显示

十进制数字的七段显示电路主要用在直接用十进制数进行输入、运算和输出的场合, 如台式计算机、计算器、计分器及其他数字式仪器仪表中, 这种逻辑电路用得很普遍。下面结

合具体例子讨论这种逻辑电路的设计方法。

1. BCD 码编码器

BCD 码编码器的输入为 10 线十进制数字,10 根输入线 D_0 、 D_1 、 $\dots$ 、 D_9 分别表示数字 0、1、 $\dots$ 、9; 输出为 4 线 BCD 码 B_8 、 B_4 、 B_2 、 B_1 ,如图 4.21 所示。

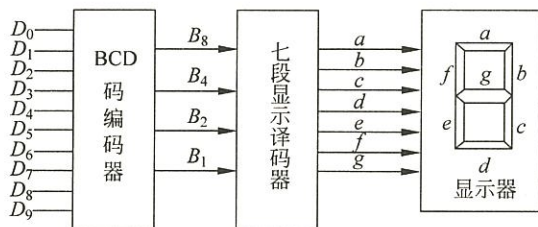


图 4.21 BCD 码编码器和七段译码器框图

BCD 码编码器的设计步骤如下。

(1) 列真值表。输入变量有 10 个: $D_0 \sim D_9$ 。由于输入变量之间是互斥的,即每次只允许一个变量取值为 1,这时输出才有确定值。列出部分真值表,如表 4.8 所示。

(2) 列函数表达式:

$$\begin{aligned} B_1 &= D_1 + D_3 + D_5 + D_7 + D_9, & B_2 &= D_2 + D_3 + D_6 + D_7, \\ B_4 &= D_4 + D_5 + D_6 + D_7, & B_8 &= D_8 + D_9 \end{aligned}$$

表 4.8 BCD 码编码器真值表

	D_9	D_8	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	B_8	B_4	B_2	B_1
0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	0	0	1	0	0	0	0	1
2	0	0	0	0	0	0	0	1	0	0	0	0	1	0
3	0	0	0	0	0	0	1	0	0	0	0	0	1	1
4	0	0	0	0	0	1	0	0	0	0	0	1	0	0
5	0	0	0	0	1	0	0	0	0	0	0	1	0	1
6	0	0	0	1	0	0	0	0	0	0	0	1	1	0
7	0	0	1	0	0	0	0	0	0	0	0	1	1	1
8	0	1	0	0	0	0	0	0	0	0	1	0	0	0
9	1	0	0	0	0	0	0	0	0	0	1	0	0	1

(3) 表达式已为最简。如果用或非门和与非门混合使用,且考虑公用部分,表达式可变换为

$$\begin{aligned} B_1 &= \overline{\overline{D_1 + D_9} \cdot \overline{D_3 + D_7} \cdot \overline{D_5 + D_7}}, & B_2 &= \overline{\overline{D_3 + D_7} \cdot \overline{D_2 + D_6}} \\ B_4 &= \overline{\overline{D_5 + D_7} \cdot \overline{D_4 + D_6}}, & B_8 &= \overline{\overline{D_8 + D_9}} \end{aligned}$$

(4) 画出逻辑图,如图 4.22 所示。

(2) 列函数表达式并化简。输出函数有 7 个, 如果按多输出函数化简法, 将是十分麻烦, 实际上, 这种情况下可接单输出函数化简, 用卡诺图法化简, 分别画出 7 个函数的卡诺图, 如图 4.23 所示。

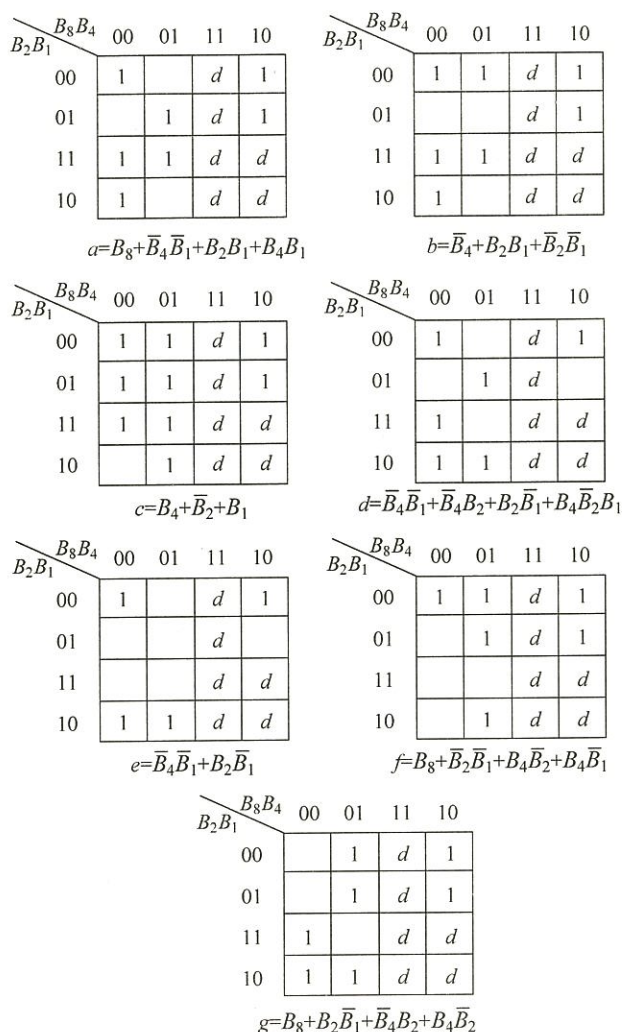


图 4.23 BCD-七段译码器卡诺图

由卡诺图得到 7 个函数的表达式为

$$\begin{aligned}
 a &= B_8 + \bar{B}_5\bar{B}_1 + B_2B_1 + B_5B_1, & b &= \bar{B}_5 + B_2B_1 + \bar{B}_2\bar{B}_1 \\
 c &= B_5 + \bar{B}_2 + B_1, & d &= \bar{B}_5\bar{B}_1 + \bar{B}_5B_2 + B_2\bar{B}_1 + B_5\bar{B}_2B_1 \\
 e &= \bar{B}_5\bar{B}_1 + B_2\bar{B}_1, & f &= B_8 + \bar{B}_2\bar{B}_1 + B_5\bar{B}_2 + B_5\bar{B}_1 \\
 g &= B_8 + B_2\bar{B}_1 + \bar{B}_5B_2 + B_5\bar{B}_2
 \end{aligned}$$

(3) 画出逻辑图。如用与非门实现上述函数, 并考虑公用部分。其逻辑图如图 4.24 所示。

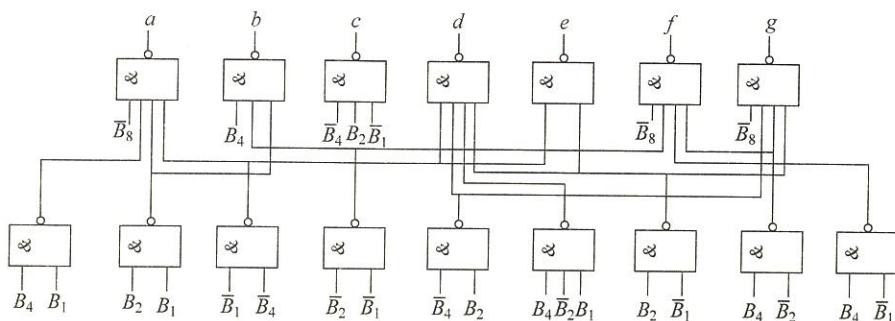


图 4.24 BCD-七段译码器逻辑图

4.5.3 二进制比较器

二进制比较器是用来完成两个二进制数的大小比较的逻辑电路,简称比较器。又称为数值比较器或数字比较器。

4.4 节已经举例说明一位和两位比较器的电路设计。这里不再赘述。

4.6 二进制译码器

所谓译码器,从广义来说就是把一种代码转换为另一种代码的逻辑电路。如前面讲过的将8421BCD码转换成七段显示器输入代码的逻辑电路就是一种译码器。这里所讲的是计算机中最常用的一种译码器,为区别于其他译码器,我们称它为二进制译码器,它在计算机中常用作指令译码、存储器地址译码等。

4.6.1 二进制译码器的功能和组成

二进制译码器有 n 个输入, 2^n 个输出。对应于每一种输入组合, 2^n 个输出中只有一个输出为1,其余全为0,或者只有一个输出为0,其余全为1。例如, $n=3$ 时,译码器的框图和真值表如图4.25所示。图中, A_1 、 A_2 和 A_3 为输入, $Y_0 \sim Y_7$ 为输出。

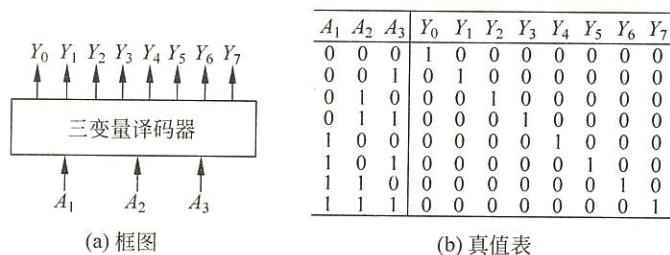


图 4.25 三变量译码器的框图和真值表

从真值表可知,对应于一组变量输入,在8个输出中只有一个为1,其余7个为0。我们把输入变量看作三位代码,而输出为8位代码,常常称为3入-8出译码器。

组成译码器的形式很多,常见的有如下几种。

1. 用与门构成的译码器

根据译码器的真值表,可写出三输入变量译码器的表达式:

$$Y_0 = \bar{A}_1 \bar{A}_2 \bar{A}_3, \quad Y_1 = \bar{A}_1 \bar{A}_2 A_3, \quad Y_2 = \bar{A}_1 A_2 \bar{A}_3, \quad Y_3 = \bar{A}_1 A_2 A_3,$$

$$Y_4 = A_1 \bar{A}_2 \bar{A}_3, \quad Y_5 = A_1 \bar{A}_2 A_3, \quad Y_6 = A_1 A_2 \bar{A}_3, \quad Y_7 = A_1 A_2 A_3$$

根据这些表达式,可画出用与门构成的译码器,如图4.26(a)所示。

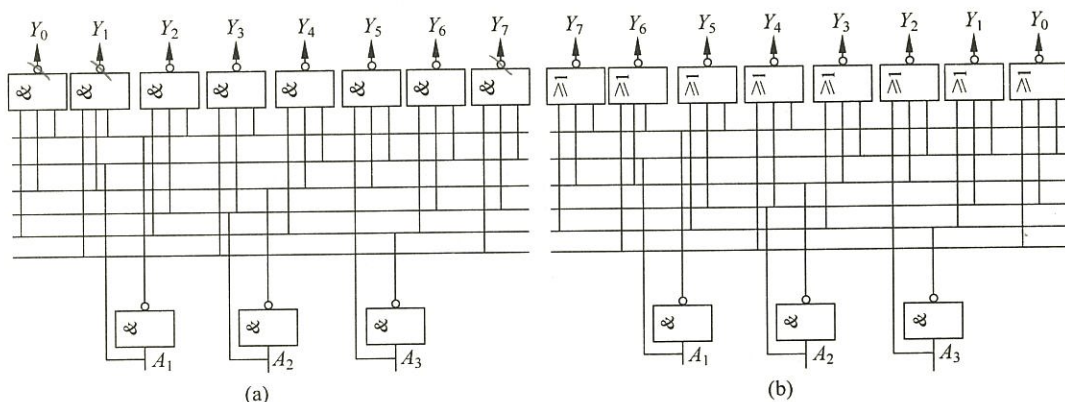


图 4.26 三变量译码器逻辑图

2. 用或非门构成的译码器

输出表达式还可写成:

$$Y_0 = \overline{A_1 + A_2 + A_3}, \quad Y_1 = \overline{A_1 + A_2 + \bar{A}_3}, \quad Y_2 = \overline{A_1 + \bar{A}_2 + A_3}, \quad Y_3 = \overline{A_1 + \bar{A}_2 + \bar{A}_3},$$

$$Y_4 = \overline{\bar{A}_1 + A_2 + A_3}, \quad Y_5 = \overline{\bar{A}_1 + A_2 + \bar{A}_3}, \quad Y_6 = \overline{\bar{A}_1 + \bar{A}_2 + A_3}, \quad Y_7 = \overline{\bar{A}_1 + \bar{A}_2 + \bar{A}_3}$$

根据这些表达式,可画出用或非门构成的译码器,如图4.26(b)所示。

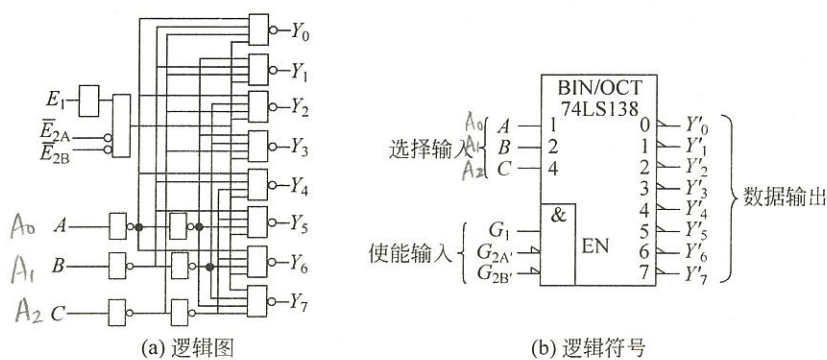
多种译码器已经做成集成电路,所以设计者不用自己做译码器,而是直接使用。

4.6.2 用中规模集成译码器进行设计

常用的中规模集成译码器有双2-4译码器74139、3-8译码器74138等。为了充分利用封装的全部引线端并增强其逻辑功能,集成译码器常常带有若干个“使能端”。“使能端”的作用有两个:一是便于扩展译码器的输入变量数;二是在“使能端”加选通脉冲可以消除由于输入倒相门的延时带来的险态。

图4.27(a)是3-8译码器74138的逻辑图。其中 E_1 、 $\bar{E}_{2A}$ 、 $\bar{E}_{2B}$ 为“使能端”。当 $E_1=1$, $\bar{E}_{2A} + \bar{E}_{2B} = 0$ (即 $\bar{E}_{2A}$ 、 $\bar{E}_{2B}$ 均为0)时,译码器处于工作状态;当 $E_1=0$ 或者当 $\bar{E}_{2A}$ 、 $\bar{E}_{2B}$ 中有一个为1时,译码器处于禁止状态。其逻辑功能列于图4.27(c)的功能表中。注意,这里是低电平输出。 $\bar{E}_2 = \bar{E}_{2A} + \bar{E}_{2B}$ 。图4.27(b)为逻辑符号,使能输入 G_1 、 $G_{2A'}$ 、 $G_{2B'}$ 相当于图4.27(a)中的 E_1 、 $\bar{E}_{2A}$ 、 $\bar{E}_{2B}$ 。

$\bar{E}_{2B}$ 。因为逻辑符号较复杂,因此下面采用逻辑框图来说明,便于大家理解。



E_1	$\bar{E}_2$	A	B	C	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7
×	1	×	×	×	1	1	1	1	1	1	1	1
0	×	×	×	×	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1
1	0	1	0	0	1	0	1	1	1	1	1	1
1	0	0	1	0	1	1	0	1	1	1	1	1
1	0	1	1	0	1	1	1	0	1	1	1	1
1	0	0	0	1	1	1	1	1	0	1	1	1
1	0	1	0	1	1	1	1	1	1	0	1	1
1	0	0	1	1	1	1	1	1	1	1	0	1
1	0	1	1	1	1	1	1	1	1	1	1	0

(c) 功能表

图 4.27 带“使能端”的三变量集成译码器 74138

下面通过例子说明译码器在设计中的应用。

【例 4.9】 用三变量译码器构成四变量译码器。

将“使能端”作为变量输入端,可以将两块三变量译码器扩展成四变量译码器,如图 4.28 所示。

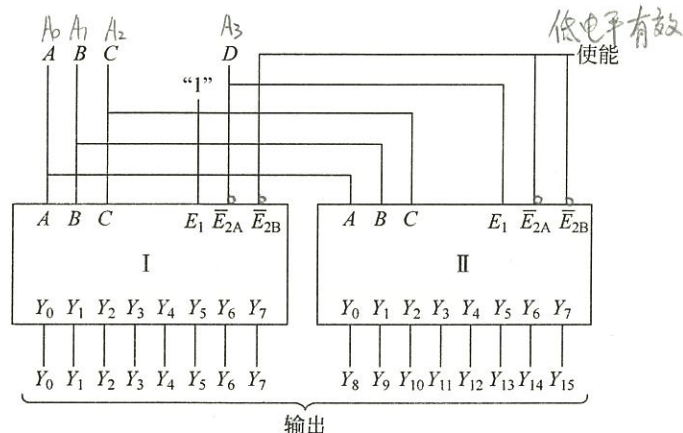


图 4.28 用三变量译码器构成四变量译码器

当输入 $D=0$ 时,片 I 工作,其输出 $Y_0 \sim Y_7$ 中有一个为“0”,其余均为“1”;而片 II 禁止,使其输出 $Y_8 \sim Y_{15}$ 均为“1”。当 $D=1$ 时,正好相反,片 I 禁止,片 II 工作,使片 I 的输出

$Y_0 \sim Y_7$ 均为“1”，而片Ⅱ的输出 $Y_8 \sim Y_{15}$ 中有一个为“0”，在图 4.28 中，“使能端” $\bar{E}_{2B}$ 可作为整个四输入变量译码器的“使能端”，用它可以两个这样的四输入变量译码器再扩展成五输入变量译码器。

因为译码器能产生输入变量的所有最小项，而任一组合函数总能表示成最小项之和的形式，即 $F = \sum m_i$ 。因此，由译码器加上或门的办法，可用来实现任何组合函数，这样做的优点是减少集成电路的使用数量。这一设计思想已被近代发展的用只读存储器实现逻辑函数所采用。

✓【例 4.10】 用译码器实现一位全加器电路。

从全加器的真值表，可以得到全加器的函数表达式为

$$\begin{cases} S_i(A_i, B_i, C_{i-1}) = \sum m(1, 2, 4, 7) \\ C_i(A_i, B_i, C_{i-1}) = \sum m(3, 5, 6, 7) \end{cases}$$

我们可以用三变量译码器的 8 个最小项输出来形成所需函数。其逻辑图如图 4.29 所示。

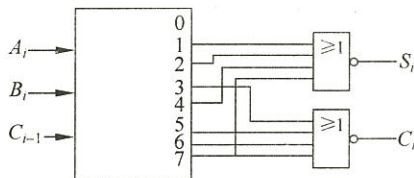


图 4.29 用译码器和或门实现的全加器

4.7 多路选择器

多路选择器又称为数据选择器，它是目前在逻辑设计中较为流行的一种通用中规模组件，它除了用作数据通路以外，还可以用来实现逻辑函数。这一节我们重点介绍用多路选择器实现函数的方法。

4.7.1 多路选择器的逻辑功能和组成

多路选择器的逻辑功能是从多个输入中选择一个，并把其信息传送到输出端，具体选择哪一个输入，则由一组选择变量确定。通常多路选择器有 2^n 根输入线， n 根选择线和一根输出线，根据 n 个选择变量的不同代码组合来选择 2^n 个不同的输入。例如，四路选择器需要两个选择变量，八路选择器需要 3 个输入选择变量。

为了实现多路选择器的功能，只要在译码器的基础上再加上一些必要的逻辑门即可。

图 4.30 是一个四路选择器的框图和逻辑图。图中， $I_0 \sim I_3$ 为 4 个数据输入端； S_1 和 S_0 为选择变量输入端。根据 S_1 和 S_0 的 4 种不同取值来控制四路数据输入。 F 为选择器的输出。

根据逻辑图很容易得到输出 F 的布尔表达式，即

$$F = I_0 \bar{S}_1 \bar{S}_0 + I_1 \bar{S}_1 S_0 + I_2 S_1 \bar{S}_0 + I_3 S_1 S_0 = \sum_{i=0}^3 I_i m_i$$

其中, m_i 为 S_1 、 S_0 组成的最小项。

同理,可以得到八路选择器的输出表达式,即

$$F = \sum_{i=0}^7 I_i m_i$$

其中, m_i 为 S_2 、 S_1 、 S_0 组成的最小项。

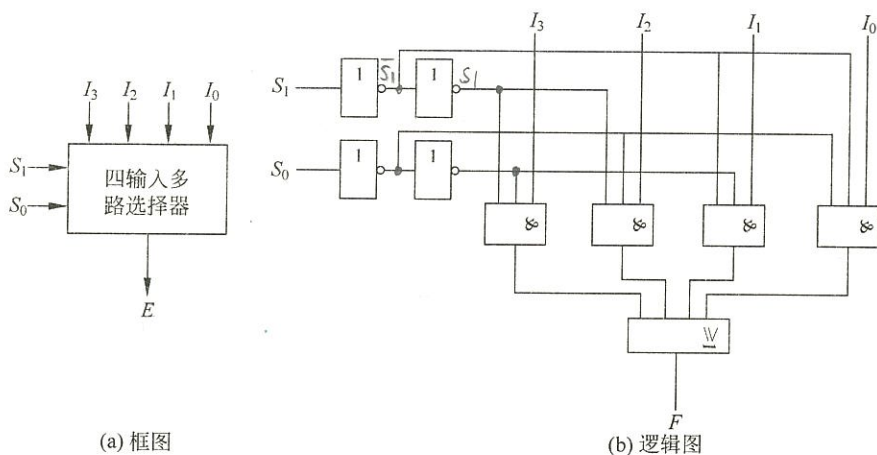


图 4.30 四路选择器

4.7.2 用多路选择器进行逻辑设计

上面讲的是用选择线 S_1 、 S_0 组成的最小项 m_i 来选择某一路输入 I_i , 这是多路选择器的基本用途。根据多路选择器的输出 F 的表达式, m_i 和 I_i 构成与项, 它们是某一与门的输入。我们能不能用输入 I_i 来选择 m_i , 以实现所需的函数 F 呢? 可以的, 因为任何函数总可以用最小项之和构成。这就是用多路选择器来实现布尔函数的基本思想。

一个二变量函数共有 16 种可能形式, 我们只要在 $I_0 \sim I_3$ 端接入不同的值 (0 或 1), 就可实现两个变量的 16 种函数。如果在 $I_0 \sim I_3$ 端接入一个逻辑变量, 其输出 F 就是一个三变量逻辑函数, 而三变量的逻辑函数可以有 256 种。这么多逻辑函数在不需另加元件的情况下, 均可由一块中规模集成电路来实现, 足见使用多路选择器来实现函数是多么灵活与方便。

用多路选择器来实现函数的设计步骤是: 首先确定选择(控制)变量; 然后求出加到每个数据输入端 I_i 的值, 它可以是常量、变量或简单函数; 最后画出逻辑图。下面通过例子来说明设计方法。

【例 4.11】用四路选择器实现异或函数。

首先以 S_1 、 S_0 作为控制变量, 然后求输入 I_i 的值。为此列真值表, 如图 4.31(a) 所示。根据真值表, 很容易找出与每一最小项 m_i 所对应的 I_i 值。例如, 最小项 $m_0 = \bar{S}_1 \bar{S}_0$ 使函数取值为 0, 故相应的 $I_0 = 0$; 最小项 $m_1 = \bar{S}_1 S_0$ 使函数取值为 1, 故相应的 $I_1 = 1$; 其余类推。

最后画出四路选择器的逻辑图, 如图 4.31(b) 所示。

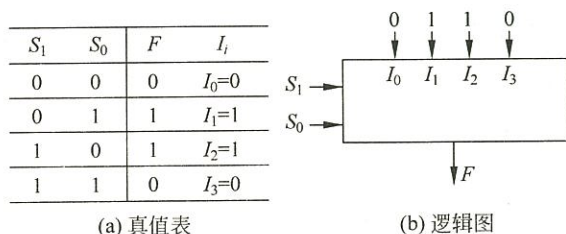


图 4.31 用四路选择器实现异或函数

【例 4.12】 用四路选择器实现三变量函数

$$F(A, B, C) = \sum m(1, 2, 4, 5) = \bar{A}\bar{B} \cdot C + \bar{A}B \cdot \bar{C} + (A\bar{B} \cdot \bar{C} + AB \cdot \bar{C})$$

首先确定控制变量, 设取 A, B 作为控制变量; 然后求输入 I_i 。列出函数的真值表, 如图 4.32(a) 所示。这里 I_i 是变量 C 的函数, 即 $I_i = f(C)$ 。

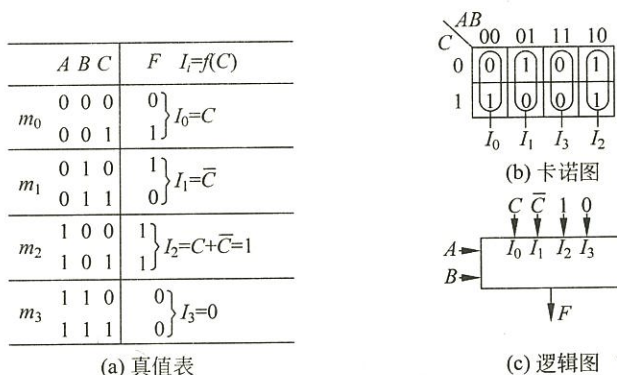


图 4.32 用四路选择器实现三变量函数

由函数的真值表可知, 在 $AB=00$ 时, 只有当 C 取 1, 才能使 $F=1$ 。根据四路选择器表达式第一项为 $\bar{A}\bar{B} \cdot I_0$, 也即只有 $I_0=C$ 时, 才使 $\bar{A}\bar{B} \cdot C=1$, 从而 $F=1$ 。同理, 可求得 $I_1=\bar{C}$, $I_2=C+\bar{C}=1$, $I_3=0$ 。

这种用真值表求 I_i 的方法称为真值表法。当变量较多时用真值表法显得烦琐, 为此, 可采用较简练的卡诺图法。函数用卡诺图表示如图 4.32(b) 所示。我们可以将函数的卡诺图看成是由 4 个子卡诺图构成的: 在控制变量 $AB=00$ 这一列, 是 I_0 的子卡诺图, $AB=01$ 列是 I_1 的, $AB=10$ 是 I_2 的, $AB=11$ 是 I_3 的。显然只要 $I_i(0, 1, 2, 3)$ 为 1, 函数 F 就为 1。用子卡诺图很容易求得 $I_i = f(C)$, 即 $I_0=C$, $I_1=\bar{C}$, $I_2=1$ 和 $I_3=0$, 其结果同真值表法。

最后画出用四路选择器实现该三变量函数的逻辑图, 如图 4.32(c) 所示。

【例 4.13】 分别用四路和八路的选择器实现函数

$$F(A, B, C, D) = \sum m(0, 3, 4, 5, 9, 10, 12, 13)$$

(1) 用四路选择器设计。

我们选取 A, B 作为控制变量(也可以选取其他两个)。

用卡诺图法求 I_i , 画出函数的卡诺图如图 4.33(a) 所示。它可以看成由 4 个子卡诺图构成, 由子卡诺图可确定 $I_i = f(C, D)$, 即

$$I_0 = \bar{C}\bar{D} + CD$$

$$I_1 = \bar{C}$$

$$I_2 = \bar{C}D + C\bar{D}$$

$$I_3 = \bar{C}$$

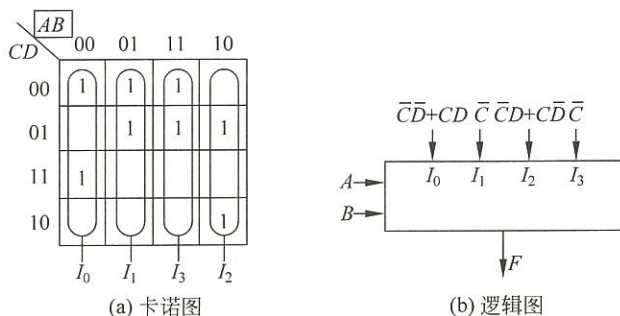


图 4.33 用四路选择器实现四变量函数

画出用四路选择器实现四变量函数的逻辑图如图 4.33(b)所示,其中 $\bar{C}\bar{D} + CD$ 、 $\bar{C}D + C\bar{D}$ 要用附加的门电路实现。

从本例可以看出,当用两个选择变量的多路选择器实现 4 个变量的函数时。附加的门电路较多。一般较经济的方案是:选择变量数为 n 时,所要实现的布尔函数变量数目不超过 $n+2$ 。

(2) 用八路选择器设计。

我们取 A 、 B 、 C 作为选择器的控制变量。

用卡诺图法求 I_i 。画出函数的卡诺图如图 4.34(a)所示。它可以看成由 8 个子卡诺图构成:相应于 $ABC=000$ 的两个方格为 I_0 ,相应于 $ABC=001$ 的两个方格是 I_1 ,其余类推。由此可求得 $I_i = f(D)$,即

$$I_0 = \bar{D}, \quad I_1 = D, \quad I_2 = 1, \quad I_3 = 0,$$

$$I_4 = D, \quad I_5 = \bar{D}, \quad I_6 = 1, \quad I_7 = 0$$

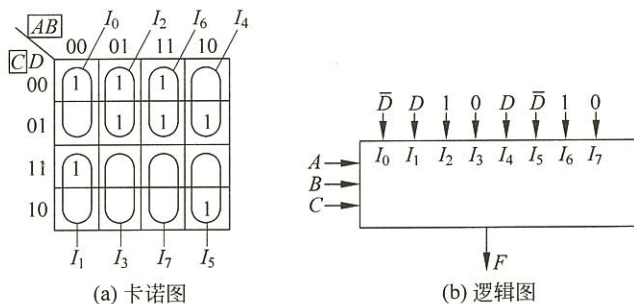


图 4.34 用八路选择器实现四变量函数

画出用八路选择器实现四变量函数的逻辑图,如图 4.34(b)所示。可见,对于四变量函数来说,用 3 个选择变量的选择器来实现较简单。

【例 4.14】 用八路选择器实现函数

$$F(A, B, C, D, E) = AB + \bar{C}D + \bar{A}E$$

对于变量较多的函数,如果仍用卡诺图法来设计就显得麻烦。这时可以直接从布尔函数表达式来求 I_i , 这就是表达式法。

我们选取 A, B 和 C 作为控制变量,根据展开定理,可以将函数展开成“积之和”的形式,即

$$\begin{aligned} F(A, B, C, D, E) = & \bar{A}\bar{B}\bar{C} \cdot f(0, 0, 0, D, E) + \bar{A}\bar{B}C \cdot f(0, 0, 1, D, E) \\ & + \bar{A}B\bar{C} \cdot f(0, 1, 0, D, E) + \bar{A}BC \cdot f(0, 1, 1, D, E) \\ & + A\bar{B}\bar{C} \cdot f(1, 0, 0, D, E) + A\bar{B}C \cdot f(1, 0, 1, D, E) \\ & + AB\bar{C} \cdot f(1, 1, 0, D, E) + ABC \cdot f(1, 1, 1, D, E) \end{aligned}$$

将此式与八路选择器的输出表达式比较可知, I_0 可以这样求得:

把 $ABC=000$ 代入原来函数中,得 $I_0 = f(0, 0, 0, D, E) = D + E$;

把 $ABC=001$ 代入原来函数中,得 $I_1 = f(0, 0, 1, D, E) = E$;

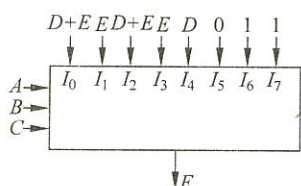


图 4.35 用八路选择器实现五变量函数

把 $ABC=010$ 代入原来函数中,得 $I_2 = f(0, 1, 0, D, E) = D + E$;

把 $ABC=011$ 代入原来函数中,得 $I_3 = f(0, 1, 1, D, E) = E$;

把 $ABC=100$ 代入原来函数中,得 $I_4 = f(1, 0, 0, D, E) = D$;

把 $ABC=101$ 代入原来函数中,得 $I_5 = f(1, 0, 1, D, E) = 0$;

把 $ABC=110$ 代入原来函数中,得 $I_6 = f(1, 1, 0, D, E) = 1$;

把 $ABC=111$ 代入原来函数中,得 $I_7 = f(1, 1, 1, D, E) = 1$ 。

画出八路选择器实现此函数的逻辑图如图 4.35 所示。

4.8 多路分配器

多路分配器的逻辑功能正好与多路选择器相反,它是将一个输入 x 分时地送到多路输出上去,具体选择哪一路输出,也是由一组选择变量确定的。通常多路分配器有一根输入线, n 根选择线和 2^n 根输出线。图 4.36 所示为一个四路分配器的框图和逻辑图。可以将多路分配器看作是译码器的一种应用,也称为译码器。

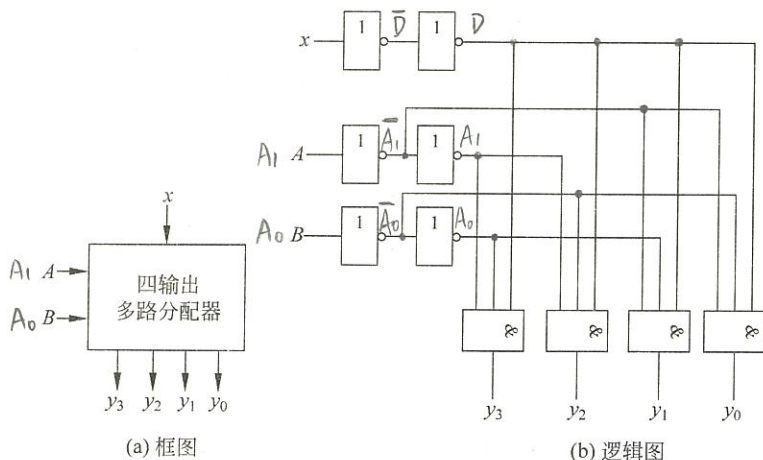


图 4.36 四输出多路分配器

综上所述,多路选择器相当于一个多路至一路的选择开关,而多路分配器相当于一个一路至多路的选择开关。如果把一个多路选择器和一个多路分配器连接起来,便可实现一条线上传送多路数据,如图 4.37 所示。在计算机的多个通用寄存器之间,往往采用这种方法提供数据通路以实现数据相互传送。

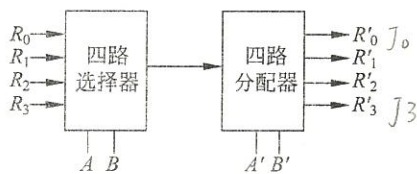


图 4.37 在一条线上传送多路数据

4.9 组合电路中的险态

布尔代数在演算过程中假设变量值保持恒定,但在实际过程中变量值会发生变化。由于电路延时就会带来潜在的差错,这会导致假信号,通常是一个小脉冲。下面要讨论的就是发生在组合电路的这种不正常的干扰信号(或称为毛刺),简称组合险态。

为了说明什么是组合险态,下面先来看两个简单的组合电路的输出情况,如图 4.38 和图 4.39 所示。

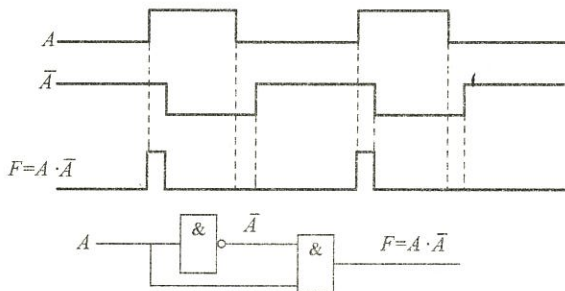


图 4.38 与门网络产生险态

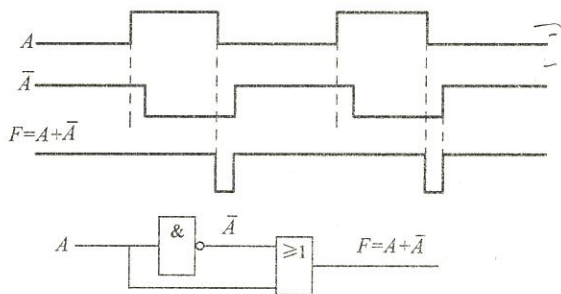


图 4.39 或门网络产生险态

如果不考虑线路的延时,则由图可写出输出函数的表达式

$$F = A \cdot \bar{A} = 0, \quad F = A + \bar{A} = 1$$

事实上,门电路都有一定的传输延时。如果考虑了门电路的延时,在网络的输出端出现了不正常的干扰信号,见图 4.38 和 4.39 中波形图。

一般地说,由于组合网络中存在门电路的延时,当某一输入发生变化时在网络的输出端可能出现瞬时的干扰信号,这种现象称为组合险态。

组合险态有静险态和动险态之分。

如果输出端在输入变化前后均具有相同稳态值,则这种险态为静险态,如图 4.38 和图 4.39 所示。静险态按照输出静态值的不同又可分为静 0 险态和静 1 险态。如果在输入变化前后输出端均为 0 值,即输出变化为 $0 \rightarrow 1 \rightarrow 0$ 的情况,称为静 0 险态,如图 4.38 所示;如果在输入变化前后输出端均为 1 值,即输出变化为 $1 \rightarrow 0 \rightarrow 1$ 的情况,称为静 1 险态,如图 4.39 所示。

如果输出端在输入变化前后具有不同稳态值,则这种险态为动险态,如图 4.40 所示。

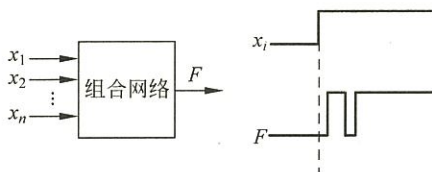


图 4.40 具有动险态的输出

由于组合险态存在干扰信号,如果把这种具有干扰的输出信号作为异步时序网络的激励输入,则可能使网络产生错误的动作。因此,如何发现和消除这种险态,这是逻辑设计中的一个重要课题。

下面介绍用卡诺图来发现和消除险态的方法。

假设有函数

$$F = \bar{A}\bar{B} + B\bar{C}$$

其相应的逻辑图和卡诺图如图 4.41 所示。

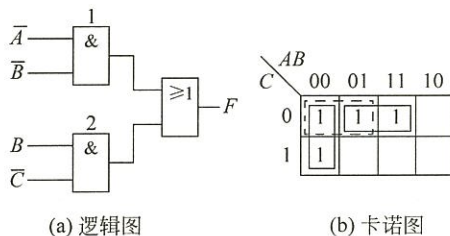


图 4.41 用卡诺图来发现险态

此函数的两个质蕴涵,在图 4.41(a)的逻辑图中,它相应于与门 1 和与门 2。当输入 $A=C=0$ 时,如果输入 B 由 1 变成 0,则函数 F 由门 2 输出“1”转成门 1 输出“1”。由于门电路的延时差异,在门 2 输出“1”到门 1 输出“1”的转换过程中, F 输出出现了险态。这一现象反映在卡诺图上,就是当 B 由 1 变成 0 时,函数从质蕴涵 $B\bar{C}$ 这个圈跨到质蕴涵 $\bar{A}\bar{B}$ 那个圈。分析这两个质蕴涵的圈可发现具有这样的特点,它们相邻但不相交。由此得到发现险态的方法:如果卡诺图中质蕴涵项的圈之间发现有相邻但不相交的情况,那么就有险态存在。

消除险态的方法是在函数中增加一个质蕴涵项 $\bar{A}\bar{C}$,即图 4.41(b)中虚线圈,它把两个相邻但不相交的圈连接在一起。这时布尔函数为

$$F = \bar{A}\bar{B} + B\bar{C} + \bar{A}\bar{C}$$

这样,当 $A=C=0$ 时,有

$$F = B + \bar{B} + 1 = 1$$

即 B 发生变化时, F 保持为 1。于是就消除了 $A=C=0$ 时, 网络的险态。

这里似乎存在一个矛盾: 为节省器材要将函数进行化简, 去掉冗余项; 而化简后, 为消除险态又要增加蕴涵项。这一矛盾该如何处理呢? 首先, 不考虑险态, 将函数化简(这是必要的); 然后, 再检查是否存在险态, 若存在险态, 则用增加适当的质蕴涵项来消除它。

以上发现和消除险态的方法是以函数的与或表达式为例。对于函数的或与表达式, 以上方法同样适用。只要在卡诺图上先求 $\bar{F}$ 的与或表达式, 如果有险态, 则增加适当的质蕴涵项来消除它; 然后再对 $\bar{F}$ 取反, 求得 F 的无险态的或与表达式。

【例 4.15】 已知函数 $F = \sum m(2, 3, 5, 7, 8, 9, 12, 13)$ 试推导无险态的“积之和”表达式及“和之积”表达式。

先用卡诺图进行函数的化简, 如图 4.42 所示。求得 F 和 $\bar{F}$ 的最简与或式为

$$F = A\bar{C} + \bar{A}BD + \bar{A}\bar{B}C, \quad \bar{F} = AC + \bar{A}\bar{B}\bar{D} + \bar{A}\bar{B}\bar{C}$$

为了消除险态, 增加质蕴涵项后, 有

$$F = A\bar{C} + \bar{A}BD + \bar{A}\bar{B}C + \bar{A}CD + \bar{B}\bar{C}D,$$

$$\bar{F} = AC + \bar{A}\bar{B}\bar{D} + \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}\bar{D} + \bar{B}\bar{C}\bar{D}$$

最后, 求得无险态的“积之和”形式为

$$F = A\bar{C} + \bar{A}BD + \bar{A}\bar{B}C + \bar{A}CD + \bar{B}\bar{C}D$$

无险态的“和之积”形式为

$$F = (\bar{F}) = (\bar{A} + \bar{C})(A + \bar{B} + D)(A + B + C)(A + C + D)(\bar{B} + \bar{C} + D)$$

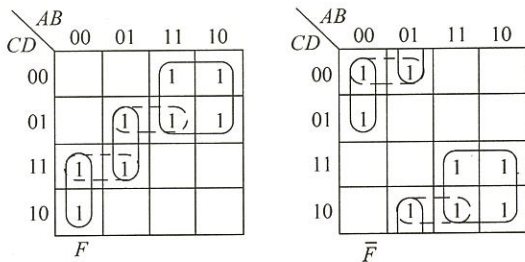


图 4.42 求函数的无险态的表达式

习题 4

4.1 分别用与非门、或非门设计如下逻辑电路:

(1) 三变量的非一致电路; (2) 三变量的偶数电路; (3) 全减器。

4.2 自选门电路设计一个比较两个 3 位二进制数 A 及 B 的电路, 要求当 $A=B$ 时, 输出 $F=1$ 。

4.3 设 A 、 B 、 C 和 D 代表 4 位二进制数码, 且 $x=8A+4B+2C+D$ 。写出下列问题的判断条件:

(1) $4 < x \leq 15$; (2) $1 \leq x \leq 9$ 。

4.4 设二进制补码 $[x]_{\text{补}} = x_0 \cdot x_1 x_2 x_3 x_4$, 分别写出下列要求的判断条件:

- (1) $(1/2 \leq x)$ 或 $(x < -1/2)$;
- (2) $(1/4 \leq x < 1/2)$ 或 $(-1/2 \leq x < -1/4)$;
- (3) $(1/8 \leq x < 1/4)$ 或 $(-1/4 \leq x < -1/8)$;
- (4) $(0 \leq x < 1/8)$ 或 $(-1/8 \leq x < 0)$ 。

4.5 设输入 $ABCD$ 是按余 3 码编码的二进制数码, 其相应的十进制数为 x , 即

$$x = 8A + 4B + 2C + D - 3 \quad 0 \leq x \leq 9$$

要求用与非门设计当 $x \leq 2$ 或 $x \geq 7$ 时, 输出 $F=1$ 的逻辑电路。

4.6 用与非门设计一个将余 3 码转换成 8421BCD 码的转换电路。

4.7 用与非门设计一个将 2421 码转换成 8421BCD 码的转换电路。

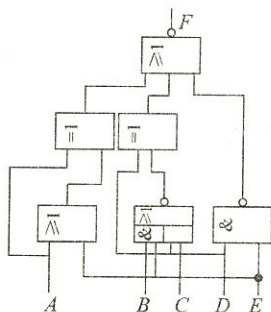
4.8 用与非门设计一个将余 3 码转换成七段数字显示器代码的转换电路。

4.9 设 x 和 y 均为 4 位二进制数, 它们分别为一个逻辑电路的输入及输出, 要求当 $0 \leq x \leq 4$ 时, $y=x$; 当 $5 \leq x \leq 9$ 时, $y=x+3$ 。试用与非门设计此电路。

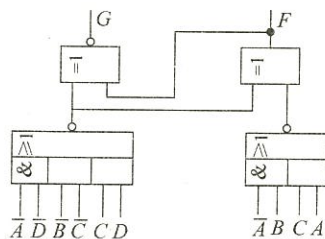
4.10 分析题图 4.10 逻辑电路, 写出其简化的逻辑表达式, 并用与非门改进设计。

4.11 已知 $[x]_{\text{原}} = x_0 x_1 x_2$, 试设计一个逻辑电路, 以原码作为输入, 要求当 $AB=00$ 时, 其输出为 $1/2$; 当 $AB=01$ 时, 其输出反码; 当 $AB=10$ 时, 其输出补码。

4.12 分析题图 4.12 逻辑电路, 写出其简化的与或表达式。

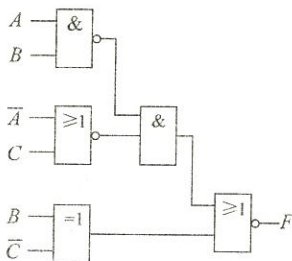


题图 4.10



题图 4.12

4.13 分析题图 4.13 逻辑电路的逻辑功能, 写出函数的表达式, 并用最简线路实现它。



题图 4.13

4.14 试为某水坝设计一个水位报警控制器, 设水位高度用 4 位二进制数提供。当水位上升到 8m 时, 白指示灯开始亮; 当水位上升到 10m 时, 黄指示灯开始亮; 当水位上升到 12m 时, 红指示灯开始亮, 其他灯灭; 水位不可能上升到 14m。试用或非门设计此报警器的

控制电路。

4.15 举重比赛有3个裁判,一个是主裁判A,两个是副裁判B和C。杠铃完全举上的裁决由每个裁判按一下自己面前的按钮来决定。只有两个以上裁判(其中必须有主裁判)判明成功时,表示成功的灯才亮。试设计此逻辑电路。

4.16 用4位全加器中规模集成电路,设计一个余3码十进制一位加法器。

4.17 用一个译码器和两个或门实现一个全减器电路。

✓4.18 用带使能端的三变量译码器构成五变量译码器。

4.19 用四路选择器实现下列函数:

✓(1) $F(A, B, C) = \sum m(2, 3, 5, 7)$

(2) $F(A, B, C) = \sum m(1, 2, 4, 7)$

(3) $F(A, B, C, D) = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$

(4) $F(A, B, C, D) = \sum m(2, 3, 4, 5, 8, 9, 10, 11, 14, 15)$

4.20 用八路选择器实现下列函数:

(1) $F(A, B, C, D) = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$

(2) $F(A, B, C, D) = \sum m(2, 3, 4, 5, 8, 9, 10, 11, 14, 15)$

(3) $F(A, B, C, D) = \sum m(0, 3, 12, 13, 14)$

(4) $F(A, B, C, D) = \sum m(0, 5, 10, 15)$

4.21 已知一函数为

$$F(x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7)$$

$$= x_0 x_6 + x_0 x_1 + x_1 \bar{x}_2 + x_0 x_3 x_4 + x_0 \bar{x}_2 + x_1 \bar{x}_6 + \bar{x}_1 x_2 + x_0 x_5 x_7 + \bar{x}_1 x_6$$

如果用多路选择器实现,则

(1) 运用多少路输入的选择器最经济?

(2) 列出所选的多路选择器的控制变量取值组合与相应的输入函数 I_i 的对应表;

(3) 画出实现函数 F 的逻辑框图。

4.22 设有如下组合函数,试推导无险态的“积之和”网络及“和之积”网络。

(1) $F(x_1, x_2, x_3, x_4) = \sum m(0, 1, 7, 15);$

(2) $F(x_1, x_2, x_3) = \sum m(0, 1, 3, 4, 7)。$

4.23 设有函数 $F(x_1, x_2, x_3, x_4) = x_1 x_3 + x_1 \bar{x}_3 x_4 + \bar{x}_1 \bar{x}_2 \bar{x}_4$

这个函数有静1险态吗? 试推导无险态的网络。

4.24 设有函数 $F(A, B, C, D) = AC + B\bar{C}D + \bar{A}\bar{C}\bar{D}$

试推导无险态的“和之积”网络。

第5章 时序逻辑电路

在组合电路中加入用来存储电路历史信息(称为状态)的存储元件即可构成时序电路。时序电路与组合电路的主要区别是：组合电路的输出只与当前的输入有关；而时序电路的输出不仅与当前的输入有关，而且还与当时的状态有关。由于时序电路的特点与组合电路不同，所以对时序电路的研究方法也与组合电路有所不同。在组合电路设计中，使用函数的真值表来描述一个组合逻辑问题；而在时序电路设计中，则使用时序机的状态表来描述一个时序逻辑问题。

时序电路分为同步时序电路和异步时序电路。同步时序电路的特点是电路有统一的时钟脉冲，在次态函数(又称为激励函数)的控制下，只有当时钟脉冲到达时电路的状态才发生改变；新的状态一旦建立，又形成新的激励，但直到下一个时钟脉冲到达之前电路的状态不会发生新的变化。同步时序电路工作稳定、可靠，而且分析和设计较为简单，这是同步时序电路被广泛采用的原因所在。但是，在有些场合，如当电路的工作速度较高或者输入是随机变化的场合，或是电路中不同部件的工作速度相差较为悬殊的场合，往往采用异步时序电路。

本章只介绍同步时序电路。

5.1 时序电路与时序机

5.1.1 时序电路的结构和特点

为了充分理解时序电路的结构和特点，不妨先回顾总结一下组合电路的结构和特点。图 5.1 展示了组合电路的结构模型，图中， $x_1, x_2, \dots, x_n$ 为某一时刻的输入； $Z_1, Z_2, \dots, Z_m$ 为该时刻的输出。显然，组合电路的逻辑功能可用下列输出函数集来描述：

$$Z_i = f_i(x_1, x_2, \dots, x_n), \quad i = 1, 2, \dots, m$$

由此可见，组合电路的特点是电路在任何时刻的输出 Z_i 仅与该时刻的输入 $x_1, x_2, \dots, x_n$ 有关，而与该时刻以前的输入无关。

时序电路是由组合电路和存储元件两部分构成的，它可以用图 5.2 所示的结构模型来表示。对图中组合电路而言，它有两组输入和两组输出。其中，输入 $x_1, x_2, \dots, x_n$ 称为时序电路的外部输入；输入 $y_1, y_2, \dots, y_k$ 称为时序电路的内部输入，它是存储元件的输出反馈到组合电路的输入；输出 $Z_1, Z_2, \dots, Z_m$ 称为时序电路的外部输出；而输出 $Y_1, Y_2, \dots, Y_p$ 称为时序电路的内部输出。



图 5.1 组合电路的结构模型

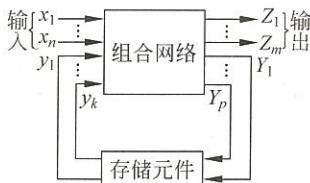


图 5.2 时序电路的结构模型

时序电路通过存储元件的不同状态来“记忆”以前时刻的输入。状态是时序电路的一个重要概念，它用来表示时序电路的过去属性。通常称 $y_1, y_2, \dots, y_k$ 为时序电路的现态，而时序电路的外部输出 Z_i 和内部输出 Y_j 是当前输入 x 和现态 y 的函数：

$$Z_i = f_i(x_1, x_2, \dots, x_n; y_1, y_2, \dots, y_k), \quad i = 1, 2, \dots, m$$

$$Y_j = g_j(x_1, x_2, \dots, x_n; y_1, y_2, \dots, y_k), \quad j = 1, 2, \dots, p$$

一般称 Z_i 为输出函数， Y_j 为激励函数或次态函数。

因此，时序电路的输出不仅与该时刻的输入有关，而且还与当时的状态有关。这就是时序电路的主要特点。表 5.1 概括了组合电路和时序电路的区别。

表 5.1 组合电路和时序电路的区别

	组合电路	时序电路
电路特性	输出只与当前输入有关	输出与当前输入和状态有关
电路结构	不含存储元件	含存储元件
函数描述	用输出函数描述	用输出函数和次态函数描述

5.1.2 时序机的定义

时序机在有的文献中亦称为有限状态机或有限自动机，它是一个从实际中抽象出来的数学模型。任何一个时间离散系统，只要满足下述定义，不管是具体的物理机器（如一个时序电路），还是抽象的虚拟“机器”（如一个算法），都可以称为时序机。

时序机是这样—个系统，它可以用 5 个参量来表征：

$$M = (I, O, Q, N, Z)$$

其中， I 为时序机的输入字母有限非空集合； O 为时序机的输出字母有限非空集合； Q 为时序机的内部状态有限非空集合； N 为时序机的次态函数，它表示输入及状态到次态的映射，即 $I \times Q \rightarrow Q$ ； Z 为时序机的输出函数，它有以下两种情况：

(1) 若输出函数 Z 是输入和状态的函数，即 $Z: I \times Q \rightarrow O$ ，那么该时序机称米利 (Mealy) 型时序机。

(2) 若输出函数 Z 仅是其状态的函数，即 $Z: Q \rightarrow O$ ，那么该时序机称穆尔 (Moore) 型时序机。

能够用时序机来描述的例子是广泛存在的。如日常生活中的电话系统、自动售票机、号码锁、数字系统中的触发器、时序逻辑电路，甚至数字计算机等都可用时序机来描述。

5.1.3 时序机的状态表和状态图

在工程应用中,时序机通常用更直观的形式——状态表和状态图来表示。所谓状态表,就是用表格方式来描述时序机的输入、状态和输出之间的关系,而状态图是用图解方式来描述上述关系,两种方法相辅相成,经常配合使用。由于两种类型时序机的状态表和状态图在表示方法上有所不同,因此下面分别进行讨论。

1. Mealy 机的状态表和状态图

Mealy 机的状态表和状态图反映了时序机的输出与它的输入以及现态之间的关系。设时序机的输入 I 为 $I_1, I_2, \dots, I_n$, 时序机的内部状态 Q 为 $q_1, q_2, \dots, q_k$, 则状态表的一般形式如表 5.2 所示。

表 5.2 Mealy 机状态表的一般形式

$Q \backslash I$	I_1	...	I_n
q_1	$N(q_1, I_1), Z(q_1, I_1)$	...	$N(q_1, I_n), Z(q_1, I_n)$
$\vdots$	$\vdots$		$\vdots$
q_k	$N(q_k, I_1), Z(q_k, I_1)$	...	$N(q_k, I_n), Z(q_k, I_n)$

表中, q_i 行和 I_j 列 ($i=1, \dots, k; j=1, \dots, n$) 相交处的项表示当时序机处于状态 q_i 并在输入 I_j 时的下一状态和输出, 这个项表示成 $N(q_i, I_j), Z(q_i, I_j)$, 这里 N 和 Z 分别是时序机的次态函数和输出函数。

状态图是这样建立的: 用小圆圈表示状态 q_i , 用有向箭头表示状态转换的方向, 并在箭头上标上 I_j/Z_k , 它表示在输入值 I_j 的情况下, 状态由 q_i 转换到下一状态 q_{i+1} 时, 其输出为 Z_k , 如图 5.3 所示。

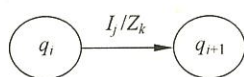


图 5.3 Mealy 机状态转换的表示

应该注意, 在状态表和状态图中没有标出时钟脉冲。而实际上, 对同步时序电路而言, 只有在时钟脉冲作用下才发生状态的转换 (这是由同步触发器的特点决定的)。这里规定: 在某时钟脉冲到达以前电路所处的状态称为电路对该时钟脉冲的现态; 而把该时钟脉冲到达之后电路的状态称为电路对该时钟脉冲的次态。还有一点要指出的是, 对 Mealy 型时序电路来说, 当输入发生变化时, 输出立即跟着变化, 而电路状态要等到下一时钟脉冲到达后才发生变化, 状态变化后, 输出再次随之发生变化。

2. Moore 机的状态表和状态图

由于 Moore 机的输出仅与现态有关, 因此在 Moore 机的状态表中, 每一现态 q_i 的行都应有相同的输出, 而与输入无关, 因此可以把它们提出来新开一列。Moore 机状态表的一般形式如表 5.3 所示。

表 5.3 Moore 机状态表一般形式

$I \backslash Q$	I_1	...	I_n	Z
q_1	$N(q_1, I_1)$	...	$N(q_1, I_n)$	$Z(q_1)$
$\vdots$	$\vdots$		$\vdots$	$\vdots$
q_k	$N(q_k, I_1)$	...	$N(q_k, I_n)$	$Z(q_k)$

在 Moore 机的状态图中, 输出 Z_k 应与状态 q_i 写在一起, 表示 Z_k 只与状态有关, 即在圆圈内标以 q_i/Z_k ; 输入仍标在箭头上, 如图 5.4 所示。



图 5.4 Moore 机状态转换的表示

由于 Moore 机的输出仅与现态有关, 因此新的输出是由转换后的状态决定的, 也就是说, 先有状态的变化, 再形成新的输出。

这一点与 Mealy 型电路有所不同, 而这一不同, 归根到底是由于两种类型时序电路的输出不同引起的。

5.1.4 完全定义机和不完全定义机

在组合电路中, 如果一个布尔函数的真值表中所有输出值都是确定的, 则此函数称为完全定义函数; 否则, 称为不完全定义函数。

在时序电路中, 如果一个时序机的状态表中所有的次态/输出都是确定的, 则此时序机称为完全定义机; 否则, 称为不完全定义机。

不完全定义机在实际中是常常遇到的。例如, 当时序机处于状态 q_i 时, 由于不可能 (或不允许) 出现输入 I_k , 所以当从状态 q_i 向下一状态转换时, 次态/输出是随意的, 或没有意义的。如下面将要介绍的基本 RS 触发器不允许同时输入置 1 和置 0 信号就是其中一例。有时, 即使是完全定义机, 往往在给其状态表的符号状态进行二进制编码时, 会使完全定义机变成不完全定义机。例如, 一个模 6 二进制加 1、减 1 计数器, 其状态表如表 5.4 所示。表中, 当 $x=0$ 时, 进行加 1 计数; 当 $x=1$ 时, 进行减 1 计数。为了表示 6 个状态 $q_0 \sim q_5$, 至少需要 3 个存储元件 (触发器), 设用 y_1 、 y_2 和 y_3 来表示。经过二进制编码后的状态表如表 5.5 所示。从表中可以看出, 由于二进制编码, 产生了两行无定义的次态和输出。这说明即使给定一个完全定义机, 经过二进制赋值后, 也会变成不完全定义机。

表 5.4 计数器的状态表

$x \backslash Q$	0	1	Z
q_0	q_1	q_5	0
q_1	q_2	q_0	0
q_2	q_3	q_1	0
q_3	q_4	q_2	0
q_4	q_5	q_3	0
q_5	q_0	q_4	1

表 5.5 经过二进制编码后的状态表

x $y_1 y_2 y_3$	0	1	Z
000	001	101	0
001	010	000	0
010	011	001	0
011	100	010	0
100	101	011	0
101	000	100	1
110	d	d	d
111	d	d	d

5.2 触发器

触发器是一种具有记忆功能、能存储二进制信息的逻辑电路,是构成时序逻辑电路的基本单元。

触发器具有两个基本特征: 第一,具有两个稳定状态,分别称为“0”状态和“1”状态,触发器的这两个稳定状态可以分别表示一位二进制代码 0 和 1。在没有外界信号作用时,触发器维持原来的稳定状态不变,即触发器具有记忆功能。触发器又称为双稳态触发器; 第二,在一定的外界信号作用下,触发器可以从一个稳定状态转变到另一个稳定状态。这表明触发器可以接收信号,并保存下来。外界信号的作用称为触发,这也是触发器名称的由来。触发器从一个稳态转变到另一个稳态的过程,称为翻转。

触发器按照逻辑功能的不同可分为 RS 触发器、JK 触发器、T 触发器和 D 触发器等几种类型。

5.2.1 基本 RS 触发器

基本的 RS 触发器是电路结构最简单的一种触发器,也是构成其他触发器的基础。由与非门构成的基本 RS 触发器的逻辑图和逻辑符号如图 5.5 所示。

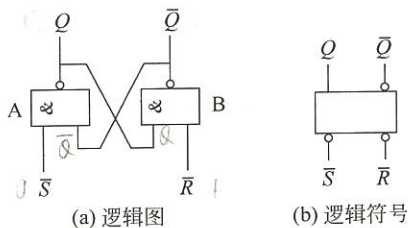


图 5.5 基本 RS 触发器

该触发器由两个两输入与非门 A 和 B 的输入/输出端交叉连接而成,它有两个输入端和两个输出端。 Q 为触发器的“1”输出端,简称“1”端, $\bar{Q}$ 为“0”端。通常将 Q 端的状态定义为触发器的状态,即当 $Q=1$ 时称触发器为“1”态;当 $Q=0$ 时称触发器为“0”态。在正常情况下, Q 与 $\bar{Q}$ 端的输出状态总是彼此互补的。当 $Q=1$ 时, $\bar{Q}=0$;反之,当 $Q=0$ 时, $\bar{Q}=1$ 。

触发器的两个输入端 $\bar{S}$ 和 $\bar{R}$ 分别称为置 1 端和置 0 端。字母 S(Set)和 R(Reset)上的一杠及图 5.5 逻辑符号中的小圆圈均表示置 0、置 1 信号是低电平有效,即平时这两个端应接高电平,只有在输入端 $\bar{S}$ 或 $\bar{R}$ 接低电平(或负脉冲)信号时,触发器的状态才可能改变。

当 $\bar{S}=0, \bar{R}=1$ 时,不论 $\bar{Q}$ 为何种状态,都有 $Q=1, \bar{Q}=0$ 。可见,不论触发器原来的状态如何,当在 $\bar{S}$ 端加低电平信号时,触发器都将被置为 1 状态。而且在 $\bar{S}$ 端低电平信号消失后,即 $\bar{S}=1, \bar{R}=1$ 时,触发器的状态保持不变($Q=1$)。

当 $\bar{S}=1, \bar{R}=0$ 时,不论 Q 状态如何,都有 $\bar{Q}=1, Q=0$ 。可见,不论触发器原来的状态如何,当在 $\bar{R}$ 端加低电平信号时,触发器都将被置为 0 状态。同理,在 $\bar{R}$ 端低电平信号消失后,触发器的状态保持不变。

当 $\bar{S}=1, \bar{R}=1$ 时,触发器状态保持不变。

当 $\bar{S}=0, \bar{R}=0$ 时, Q 与 $\bar{Q}$ 全为 1,使 Q 与 $\bar{Q}$ 彼此互补的逻辑关系遭到破坏。这是不正常情况,是不允许的。而且,当 $\bar{S}$ 和 $\bar{R}$ 低电平信号同时消失后,触发器的状态将无法确定,可能是 0,也可能是 1。

表 5.6 为基本 RS 触发器的真值表。其中, $\bar{S}$ 和 $\bar{R}$ 为触发器的输入信号; Q^n 为触发器接收信号之前的状态,称为现态; Q^{n+1} 为触发器接收信号之后的状态,称为次态。

表 5.6 基本 RS 触发器真值表

Q^n	$\bar{S}$	$\bar{R}$	Q^{n+1}
0	0	0	×
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	×
1	0	1	1
1	1	0	0
1	1	1	1

从使用触发器的角度出发,通常用简化的真值表,即功能表,表示一个触发器所能完成的功能。基本 RS 触发器的功能表如表 5.7 所示。

表 5.7 基本 RS 触发器功能表

$\bar{S}$	$\bar{R}$	Q^{n+1}	功能
0	0	不正常	不允许
0	1	1	置 1
1	0	0	置 0
1	1	Q^n	保持

根据基本 RS 触发器的真值表,可以得到次态 Q^{n+1} 与现态 Q^n 、输入 $\bar{S}$ 及 $\bar{R}$ 的逻辑关系表达式为:

$$\begin{cases} Q^{n+1} = S + \bar{R}Q^n \\ RS = 0 \quad (\text{约束条件}) \end{cases} \quad (5.1)$$

式中, $RS=0$ (或 $\bar{S}+\bar{R}=1$) 表示不允许同时出现 $\bar{S}=0, \bar{R}=0$ 的情况,称为约束条件。式(5.1)常称为触发器的特征方程,也称为状态方程或次态方程。

在时序电路设计中,往往已知触发器的现态 Q^n 与次态 Q^{n+1} ,要求所需的输入信号值,称为输入激励。将不同现态和次态时的输入激励列成表格,称为触发器的激励表。表 5.8 为基本 RS 触发器的激励表。

表 5.8 基本 RS 触发器激励表

Q^n	Q^{n+1}	$\bar{R}$	$\bar{S}$
0	0	1	1
0	1	1	0
1	0	0	1
1	1	1	0

CLK 高电平

5.2.2 同步 RS 触发器 → 电平触发的触发器

实际的数字系统工作时,要求各部分电路要协调动作,因此系统中用一个同步信号来指挥电路各部分协调动作,该信号又称为时钟脉冲信号,简称时钟,用 CP (Clock Pulse) 表示。电路中的触发器只有在时钟脉冲到来时,才按照输入信号改变状态。这种受时钟控制的触发器统称为同步触发器或钟控触发器。

同步 RS 触发器的逻辑图和逻辑符号如图 5.6 所示。该触发器由 4 个与非门构成,门 A 和门 B 构成基本 RS 触发器,门 C 和门 D 为控制门,CP 为控制信号。 S 和 R 两个输入端分别是置 1 端和置 0 端,高电平有效。在逻辑符号图中,CP 输入端有标记“ $\wedge$ ”,表示该输入端为脉冲信号。

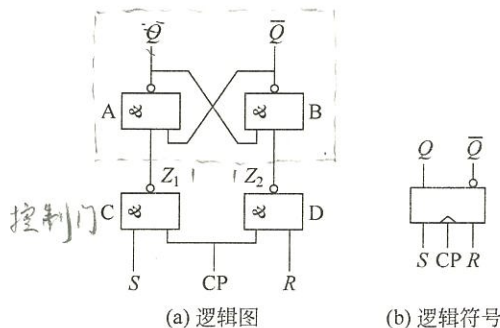


图 5.6 同步 RS 触发器

当 $CP=0$ 时,门 C 和门 D 均输出高电平,门 A、B 构成的基本 RS 触发器处于保持状态。
当 $CP=1$ 时,门 C、门 D 的输出 Z_1, Z_2 为:

$$Z_1 = \overline{S \cdot \underbrace{CP}_1} = \overline{S} \quad (5.2)$$

$$Z_2 = \overline{R \cdot CP} = \overline{R} \quad (5.3)$$

同步 RS 触发器的功能表如表 5.9 所示,和表 5.7 完全相同,只是增加了“CP=1 时有效”这个条件。因此,同步 RS 触发器的特征方程与基本 RS 触发器的完全相同。

表 5.9 同步 RS 触发器功能表(CP=1 时有效)

✓ S	R ✓	Q^{n+1}	功能
0	0	Q^n	保持
0	1	0	置 0
1	0	1	置 1
1	1	不正常	不允许

5.2.3 JK 触发器

在 CP 操作下,根据输入信号 J、K,具有置 0、置 1、翻转和保持功能的电路,称为 JK 触发器,其逻辑符号和时序图如图 5.7 所示。JK 触发器的逻辑功能如表 5.10 所示。

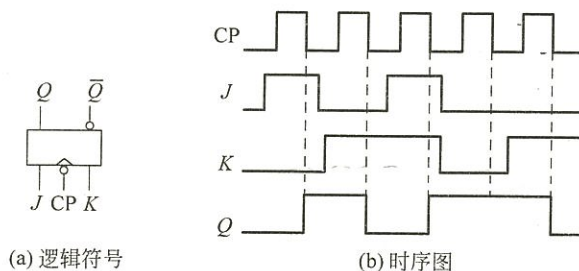


图 5.7 JK 触发器

表 5.10 JK 触发器的次态真值表

J	K	Q^n	Q^{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

其特性方程为

$$Q^{n+1} = J \overline{Q^n} + \overline{K} Q^n \text{ (CP 下降沿到来后有效)} \quad (5.4)$$

JK 触发器的激励表如表 5.11 所示。

表 5.11 JK 触发器的激励表

Q^n	Q^{n+1}	J	K
0	0	0	d
0	1	1	d
1	0	d	1
1	1	d	0

5.2.4 D 触发器

在 CP 操作下, 根据输入信号 D , 具有置 0、置 1 功能的电路, 称为 D 型触发器。根据对维持阻塞 D 触发器电路分析, D 触发器的逻辑功能如表 5.12 所示。

表 5.12 D 触发器的次态真值表

D	Q^n	Q^{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

其特性方程为

$$Q^{n+1} = D \quad (\text{时钟 CP 变成高电平, CP 上升沿到来后有效}) \quad (5.5)$$

表 5.13 为 D 触发器的激励表。

表 5.13 D 触发器的激励表

Q^n	Q^{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

5.2.5 T 触发器

在 CP 操作下, 根据输入信号 T 的不同, 具有保持和翻转功能的电路, 称为 T 型触发器, 其逻辑符号和时序图如图 5.8 所示。T 触发器的逻辑功能如表 5.14 所示。

表 5.14 T 触发器的次态真值表

T	Q^n	Q^{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

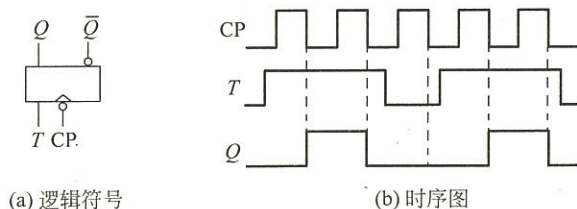


图 5.8 T 触发器

T 触发器是由 JK 触发器演变而来,即 J 、 K 相连作为信号输入端 T 。因此其特性方程也可以由 JK 触发器的特性方程得出

$$\begin{aligned}
 Q^{n+1} &= J \bar{Q}^n + \bar{K} Q^n \\
 &= T \bar{Q}^n + \bar{T} Q^n \\
 &= T \oplus Q^n \text{ (CP 下降沿到来后有效)}
 \end{aligned} \tag{5.6}$$

T 触发器的激励表如表 5.15 所示。

表 5.15 T 触发器的激励表

Q^n	Q^{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

5.3 同步时序电路的分析与设计

任何一个数字逻辑电路,总是从一组给定的输入,得到一组确定的输出。在组合电路中,多次重复出现的输入,可得到完全相同的输出。但是在时序电路中,如果一组输入多次重复出现,电路的输出却不尽相同,这是因为在同一输入的情况下,可能有不同的现态。因此,时序电路的分析与设计要比组合电路的分析与设计复杂得多。

设计一个同步时序电路,一般可按如下步骤进行。

(1) 根据逻辑问题的文字描述,建立原始状态表。进行这一步时,可先借助于原始状态图,再构成原始状态表。这一步得到的状态图和状态表是原始的,其中可能包含多余的状态。

(2) 采用状态化简方法,化简原始状态表。这一步得到一个用字符表示状态的简化状态表。

(3) 进行状态分配(或状态赋值),即给予简化状态表中每个符号状态以二进制代码表示,这一步得到一个二进制状态表。

(4) 根据二进制状态表和选用的触发器的激励表,求电路的激励函数和输出函数。

(5) 根据激励函数和输出函数表达式,画出所要求的逻辑图。

同步时序电路的分析过程与设计过程正好相反,其主要步骤如下。